Perch: Bounded, Recoverable Grants for Soroban Smart Accounts

Willem Wyndham and Elizabeth Engelman

The Aha Company

Abstract. Soroban smart accounts let an owner give applications bounded authority, but a grant's meaning must survive nested contract calls, rule selection by whoever submits the transaction, and account recovery. We present Perch, a smart account in which the owner installs grants: rules that fix a target contract, functions, argument conditions, a signer threshold, an optional expiry and an optional sliding-window spending cap. A pure evaluator linked into the account decides each authorization from the selected rules, their spending records and a grant generation, before any signer is authenticated. We translate this Rust evaluator into Lean with Charon and Aeneas and prove that the translation agrees with a model on every input, that acceptance implies permission, and that histories accepted from empty records keep each cap window within its limit. An account model shows that after a completed recovery no earlier grant authorizes; the account shell is tested, not proved. We compare Perch with oz-policy-builder, which attaches a shared predicate interpreter to OpenZeppelin accounts, run one task in both designs, and evaluate Perch on a local Soroban host and on Stellar testnet.

Keywords: Smart accounts · Authorization · Soroban · Capabilities · Formal verification · Lean

1 Introduction

A player keeps a Perch account in Nido and wants Guess the Number, a Soroban game that takes 1 XLM per wrong guess and pays out its balance on a right one, to play without a passkey prompt per turn. The owner can install a *grant* naming the contract and functions an app may call, the signer that may act, how much it may spend over time and when it ends; it must not let the app reach the account's own entry points or its recovery controller. In the reference run (§10), grant 1 lets an app transfer at most 10 XLM in any 100-ledger window (about eight minutes at the nominal 5-second ledger), and grant 3 lets the app key call the game's `guess` and lets the game take at most 2 XLM per 100 ledgers from the account, so two lost guesses fit and a third is refused.

Problem and approach. A grant needs one meaning across the nested calls of one authorization, which Soroban asks the account to decide one at a time; under rule selection, which whoever builds the transaction performs; and across recovery,

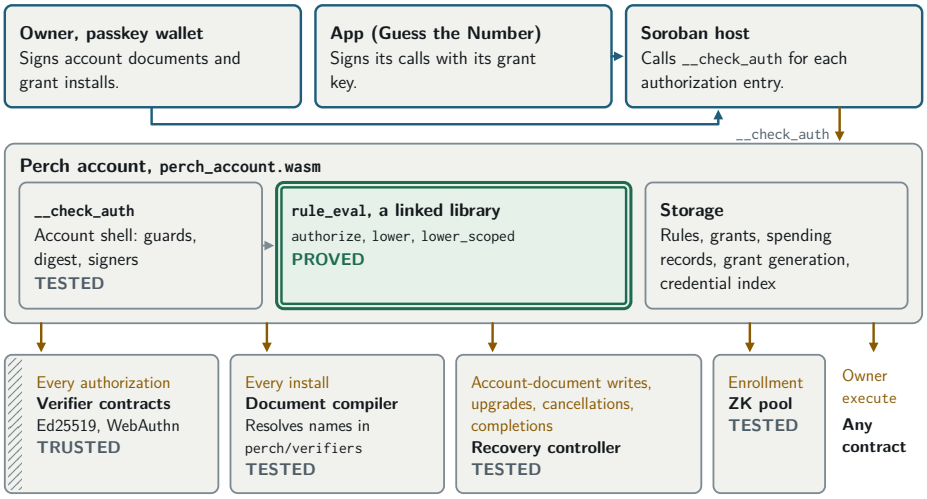


Fig. 1: The parts of a Perch account at the pinned build. Arrows are calls out of the account, labelled with when they happen. Double border: the evaluator functions the theorems cover. Single border: tested code. Hatched: trusted components.

which replaces the credentials a grant was issued under. A cap also needs state. Perch makes the decision a pure state transition, linked into the account and run before any signer is authenticated, and separates what is proved about it from the obligations of the account around it (Fig. 1).

Contributions.

1. A Soroban smart-account design whose authorization is a pure state transition over the selected rules, account-held spending records and a grant generation. Grant lowering excludes the account, its adopted recovery controller, and the zero-knowledge adapter and pool of its recovery configuration (§§5–6).
2. A grant generation that a completed recovery advances: afterwards no earlier grant authorizes, nor, in the account model, any grant derived from one, and a credential index bars their keys from later documents other than a completion's target (§7).
3. Lean theorems over the Aeneas translation of the Rust evaluator, with an explicit trust base and 57 Lean-generated traces that connect them to the account (§9).
4. A source-cited comparison with oz-policy-builder, with one task run in both designs (§8), and an evaluation on a local Soroban host and on testnet (§10). Comparative cost measurement is proposed.

Research questions. **RQ1**: does the built account enforce grants, caps and recovery invalidation as designed? **RQ2**: which parts of a decision are proved, tested or trusted? **RQ3**: what does a decision cost on chain, for Perch alone? **RQ4**: how does a linked evaluator differ from an attached shared interpreter? §10 answers

RQ1 and RQ3, §9 RQ2 and §8 RQ4. The design is an open pull-request stack, previewed on Stellar testnet (§4).

2 Background: Soroban Authorization and OpenZeppelin Accounts

Soroban authorization. A contract that needs an address's approval calls `require_auth`; for a contract account, the host calls the account's `__check_auth` [15,26]. A transaction carries one authorization entry per approving address, except CAP-0071 delegates [16], whose approval rides in the account's entry through `delegate_auth`. The entry's signature payload hashes the network ID, a nonce, a signature-expiration ledger and the entry's invocation tree. The host passes the account the payload, the signatures and the entry's *contexts*: every call and contract creation in the tree, in preorder, root first. The account must decide every context. The nonce is consumed only when the transaction succeeds, so a refused entry stays usable until its expiration ledger. A lost guess gives two contexts, `game.guess` and the `xlm.transfer` it makes from the account, each decided by its own selected rule (Fig. 6).

OpenZeppelin accounts. OpenZeppelin's Soroban library [19], which oz-policy-builder builds on and whose authorization types Perch reproduces, stores *context rules*: a scope (`Default`, `CallContract` or a contract-creation scope), a name, an optional `valid_until` ledger, signers and policies. A signer is *external* (a verifier contract and key material) or *delegated* (another address). OpenZeppelin v0.8.0 checks a delegated signer with `require_auth_for_args`, in an entry of its own; Perch checks one through CAP-0071, inside the account's entry. A *policy* is a contract with `enforce`, `install` and `uninstall` hooks. The payload names one rule per context (`AuthPayload.context_rule_ids`). `__check_auth` validates the selected rules, refuses a signer on none of them, authenticates the signers and calls each policy's `enforce` in the rule's stored order: address order for the policies it was created with, then any added later. Perch uses the same payload field; it differs in what decides.

3 Threat Model and Goals

The adversaries (Table 7) are *A1*, a grantee that signs within rules naming its key and builds any invocation tree; *A2*, a submitter that wraps signed entries and resubmits refused ones until they expire; *A3*, a thief of the owner key; *A4*, guardians below quorum; and *A5*, the `perch/verifiers` registry's manager or admin. Trusted, not adversaries: the Soroban host and validators, the verifiers documents name, the adopted recovery controller, the document compiler, rustc 1.97.1, the wallet's display and, for the proofs, the trust base of §9. In Saltzer and Schroeder's terms [23], `__check_auth` mediates every `require_auth` on the account that the host satisfies from an authorization entry; one satisfied by invoker authority, as in the owner's `execute`, never reaches it (§12).

Goals. G1, containment: no grant rule allows a call to the account or its excluded set; proved for lowering under `Doc.Valid` (T8, T9), runtime recheck tested. *G2, permission soundness*: an accepted authorization is allowed by the rules it selects; proved for the evaluator (T2); signer authentication, selection and context conversion tested. *G3, window safety*: no capped rule's window exceeds its limit; proved for the evaluator from empty records (T6); selection and record persistence tested. *G4, recovery invalidation*: after a completed recovery no earlier grant authorizes through the account; proved in the account model (T10), tested, executed and recorded on testnet. *G5, signature binding*: an external signer's signature binds the account, the entry and the selected rule IDs; tested, with a paper lemma in Appendix B; delegated signers excluded. *G6, fail-closed refusal*: a refusal writes nothing; tested. *Non-goals*: budgets across grants; effects outside authorization (allowances, the owner's `execute`, funds held by other contracts); confidentiality; verifier correctness; and what a wallet displays.

4 Implementation Status

The design is pinned at Perch f0275f4, the head of an unmerged pull-request stack ending at #68, with its evaluator at rule-core cbabe59. Perch's manifests record an earlier design, account 0.5.1 built from c135aca, on testnet and, since 2026-10-26, on mainnet [20]; it decides with a fork of OpenZeppelin's account library and two Perch policy contracts, and its accounts cannot be upgraded in place to this design. This paper's contracts run on a separate testnet deployment, where a preview of Nido creates accounts. No external audit of either Perch build has been completed. Table 2 gives the status of each component.

5 Grants, Rules and Spending Caps

An *account document* describes the owner's policy, including administrative rules and recovery; a *grant document* describes a bounded set of rules for one application. Applying a grant document creates or replaces an *installed grant*, the rules the account consults, stored under a grant ID and stamped with the current grant generation. Grant 1 as the reference run installs it, abbreviated:

```
{ "version": 1, "network": "Standalone Network ; February 2017",
  "signers": [ { "id": "app", "verifier": "C...ED25519", "key": "...Ed25519 key, hex..." } ],
  "rules": [ { "name": "spend", "scope": { "type": "contract", "address": "C...XLM" },
               "principals": { "type": "all", "signers": ["app"] }, "functions": ["transfer"],
               "cap": { "limit": "100000000", "period-ledgers": 100 } } ] }
```

`scope` is the rule target and `principals` the signer condition; `cap.limit` is 10 XLM in stroops, charged against the amount argument of `transfer(from, to, amount)`, the only function a capped rule may list. A rule names one target contract, a function list, argument conditions and a threshold over listed signers, and optionally an expiry and a cap. Its five kinds of argument condition, `is-self`, `address-eq`, `string-in`, `string-prefix`, `u32-eq`, are conjunctive, and a document has no negation. A document cannot bound a single call's amount: a capped rule gets only a type guard that argument 2 is an `i128`.

Installation. The owner authorizes `apply_grant` (exported as `apply_scope`; Table 4). The stateless document compiler validates the JSON, resolves verifier names through the `perch/verifiers` registry and returns canonical bytes, their hash `doc_hash` and a model. The account lowers the model with rule-core's `lower_scoped` under its address, its excluded set and the current generation, the step the lowering and containment theorems cover. Grant lowering refuses (`ScopeEscalation`) a document that configures recovery, and any rule whose target is the account, its adopted recovery controller, or the zero-knowledge adapter and pool of its recovery configuration. The account also refuses non-normal documents, rules with no signer, and credentials that recovery revoked or a stale grant named, then stores the grant; a client can rebuild its JSON from chain state and check it against `doc_hash` (Appendix B).

Rule semantics. A rule set denotes the union of its rules' permissions, but the account does not search it: the transaction names one rule per context, which must itself allow the context, be unexpired and, if it carries a generation, be current. A supplied signer in no chosen rule is refused. rule-core's 10 attenuation steps each return a rule set allowing nothing the original does not (T4, T5; Appendix B); no account entry point installs an attenuated grant at the pin.

Sliding windows. For period P, the window at ledger L holds the charges made after $L-P$ and through L (Fig. 4). The evaluator checks the new total against the window and returns a new record; contexts selecting one rule in one authorization see each other's charges, and a negative or malformed amount, overflow, a ledger before the newest charge and a full history fail closed. The 65,536-byte entry limit holds 907 spends per record, so a grantee can exhaust its own window with small transfers.

6 The Authorization Path

The account decides with rule-core's `rule_eval::authorize` (D3, §9), which neither searches for rules nor verifies signatures. Figure 2 lists the steps of `__check_auth`. A recovery completion leaves at step 4 for the recovery controller, so a recovery completion never depends on the evaluator; enrollment and reconfiguration are document writes the evaluator decides. Otherwise the account binds the payload, resolves the selected rule IDs, runs the evaluator, authenticates every claimed signer and stores each changed record, so a refused batch calls no verifier. A stale grant refuses with `StaleScope` (auth error 3) and any other evaluator refusal with `RuleEvalRefused` (5), inside `Error(Auth, InvalidAction)`; a wallet re-runs the evaluator off chain for the reason. Because a controller can be adopted after a grant is installed, step 4 rechecks each selected grant rule against the current excluded set (`ScopeEscalation`, 4); the install-time property is proved (T8), the recheck tested. Infrastructure addresses derive from a Wasm hash and registry path fixed at build time; once that code is deployed on a network, no registry action changes it (Appendix B).

The Soroban host calls `__check_auth(signature_payload, auth payload, contexts)`; the auth payload carries `context_rule_ids` and the claimed signers.

No.	Step	Exit and code	Assurance
1	Refuse if any context calls a reserved hook name (`install`, `uninstall`, `enforce` and seven `rcv_*` names) on any contract.	`ReservedFunction` (1)	tested
2	Classify a recovery completion: one `apply_account_doc` context (exported `apply_doc`) on this account, selecting the recovery rule.	–	tested
3	Refuse while an authorized `Protected` attempt freezes the account, unless this is its completion.	`AccountFrozen` (2)	tested
4	Refuse a selected grant rule that calls the recovery controller or its zero-knowledge adapter or pool. A completion leaves here: the controller's `rcv_complete` checks the approved document bytes and the completion window.	`ScopeEscalation` (4); a completion: decided by the recovery controller	tested
5	bound = SHA-256(domain ‖ account ‖ signature payload), with domain perch/account/auth-addr-v1.	–	tested
6	Resolve the selected rule IDs to stored rules and spending records; convert the contexts.	`ContextRuleNotFound` (3000); `RuleEvalRefused` (5)	tested
7	`rule_eval::authorize` maps rules, records, contexts, choices, signers, ledger and generation to the next records. It reads no storage and checks no signature.	stale grant: `StaleScope` (3); otherwise `RuleEvalRefused` (5)	proved
8	digest = SHA-256(bound ‖ XDR(rule IDs)). Authenticate each signer: an external one through its verifier contract's `verify(digest)` (trusted), a delegated one (CAP-0071) through the host's `delegate_auth`, which does not bind the rule IDs. A verifier that returns false gives `ExternalVerificationFailed` (3003).	bad signature: the verifier traps (`Error(Crypto, InvalidInput)` for Ed25519); the host reverts	tested
9	Store each changed capped spending record. Authorized.	–	tested

Fig. 2: One authorization through `__check_auth` at the pinned build. Step 7, the evaluator, is covered by the theorems; the other steps are tested account-shell code. Each exit except a verifier trap names the account's error and its number, which a wallet finds in the diagnostic events of the host's `Error(Auth, InvalidAction)`; a verifier that traps fails in the host's signature check. No exit leaves a write. A recovery completion leaves at step 4 and is decided by the recovery controller.

What a signature binds. For a legacy `Address` credential the host's payload P does not name the account; `AddressV2` and delegated entries hash `SorobanAuthorizationWithAddress`, which adds it. Perch binds the account either way: an external signer signs $D = H(H(\tau \,\|\, id_A \,\|\, P) \,\|\, \mathrm{XDR}(\rho))$, with $H = \mathrm{SHA\text{-}256}$, the tag $\tau =$ `perch/account/auth-addr-v1`, the account's ID id_A and the selected rule IDs ρ. A delegated signer approves the host's address-bound payload, which omits ρ, so a submitter may select any rule that lists the delegate; OpenZeppelin v0.8.0 binds ρ for both signer kinds (Table 8). A credential's authority is the union of every live rule that names it; binding the rule IDs stops a relayer, not the signer, from choosing among those rules.

Ed25519 [14] and WebAuthn [29] verifiers are deployed and an ML-DSA-65 [18] verifier is built and tested (Appendix B).

7 Recovery and the End of Authority

The account document's recovery profile states the threat recovery must handle. *Loss* assumes the owner's key may be lost, not stolen: activity continues during an attempt, and the owner, or a thief holding the key, can cancel it. *Protected* assumes theft: once the recovery condition authorizes an attempt, the account refuses everything but its completion, and changing recovery or upgrading needs the condition's evidence. A Protected configuration may enroll a baseline, the hash of an approved document; a compromise recovery restores it and revokes every credential added since. The condition is a guardian quorum, a zero-knowledge proof, or both (modes `guardian-only`, `zk-only`, `combined`; Appendix B).

Anyone may open an attempt naming replacement credentials; the controller derives the target document on chain, so nobody chooses a target hash. Each guardian approves the SHA-256 digest of a statement binding the network, account, controller, configuration, timing, attempt and the hashes of source, target and replacement set. With the reference run's 2 guardians, the second approval authorizes the attempt at ledger A, and completion is allowed in [A + 10, A + 110). Completion is one transaction: the controller's `rcv_complete` confirms the approved bytes and the window, and `rcv_sync` marks the attempt completed, spends any nullifier and advances the controller's epoch. The account applies the target, advances the grant generation (refusing on overflow), permanently revokes the administrative credentials the target replaced or removed, lifts any freeze and drops any pending upgrade (Fig. 3).

Ending authority. Each earlier grant keeps its earlier generation, so it refuses every nonempty authorization while its rules remain stored. A credential index refuses (`WipedCredential`) any later document naming a credential an earlier grant named, except the recovery target, which may keep it until rotated. Table 6 separates invalidation, a policy event, from collection, a storage operation, and archival, which is neither.

8 Comparison with oz-policy-builder

oz-policy-builder [28] (OZ-PB) adds rules to OpenZeppelin accounts: an off-chain synthesiser (`@crediolabs/policy-synth`) records a transaction or takes a declared Intent and writes a predicate, and one shared `policy-interpreter`, attached to a context rule as a policy, evaluates it in `enforce` on every guarded call. Perch's deployed build c135aca used this attached-policy shape; the design here moves the decision into the account. We compare at Perch f0275f4, OZ-PB 43b13ed (v5.2.0 and 17 later commits, none under its contracts) and OpenZeppelin v0.8.0, reading comparator design claims from source and public documentation. Each claim states what it rests on (source, a run, tests or a proof); these are evidence about different objects, and we do not rank them. Appendix C gives twenty dimensions with sources and the figures cited here.

Per-call bounds and limits over time. The interpreter decides each call alone: it does not see what a transaction moved, keep rolling totals or count calls (OZ-PB `docs/stellar.md:404-407`). The synthesiser writes a per-call bound (`lte`); a limit over time comes from OpenZeppelin's optional `spending_limit`, attached on `transfer` only, or from a token allowance. On a local network, an OpenZeppelin account whose rule carried only the interpreter, with a 5-XLM per-call bound, applied three sends that moved 9 XLM, as documented (Fig. 7). In Perch the window is part of the rule, charged in the decision that checks the call, and T6 bounds every window from empty records; a document cannot bound a single call's amount.

Authoring. OZ-PB starts from what an app did: it synthesises the least predicate permitting a recorded call or an Intent, verifies it by dry-running the deployed interpreter's own `check`, renders a review and returns unsigned install bytes. Perch starts from what the owner grants: a declared document, compiled on chain and lowered against the account's own authority (§5). The approaches are complementary; a recording front end for Perch is future work (Fig. 9 spells one intent both ways).

Evidence. OZ-PB builds at 43b13ed, and its 575 Rust test runs and 1,823 TypeScript tests pass in our runs. A policy is verified by simulating the deployed interpreter, so one evaluator verifies and enforces; 26 of its 440 interpreter tests run against OpenZeppelin's v0.8.0 account, and it keeps a STRIDE register. An external audit of v5.2.0 has been in progress since 26 October 2026, and its off-chain packages were hardened after an external penetration test (pull request #78, branch `fix/hacken-pt-findings`). Perch's evaluator is proved in Lean over its Aeneas translation, tied to the Rust by extraction and to the account by 57 traces; the theorems do not reach the Wasm, the shell is tested, and no audit has been completed.

Contexts, composition, lifecycle, binding. Both judge each context under its own rule (OZ-PB `docs/architecture.md:307-309`; Fig. 6); OZ-PB's execution adapter lets one predicate relate the calls of a batch (`docs/stellar.md:191-195`). An OpenZeppelin account composes a decision from the interpreter, a threshold policy and `spending_limit`; with the interpreter alone any one authenticated signer suffices (policy-synth README, "Threshold"). A Perch grant is one document decided by one evaluator. The interpreter pins a policed rule to its exact signer set (error 204 after any change), and removing a rule with only rule 0's signatures leaves its interpreter entry until it archives (`docs/stellar.md:548-562`); in Perch a completed recovery stops every earlier grant from authorizing (T10). OpenZeppelin v0.8.0 binds the account and rule IDs for both signer kinds; Perch's delegated signers do not bind rule IDs (Table 8).

The walkthrough run. In one soroban-sim harness both designs gave the same verdict on each of seven requests (Table 1). The OpenZeppelin account called `spending_limit` and the interpreter, and its delegated signer approved in a second authorization entry; Perch called its verifier. The signer kinds differ, because OZ-PB takes only delegated signers. Over the cap, `spending_limit`, first by address here, refused before the interpreter ran; Perch refused before any verifier call. This is one task, not a cost comparison; W1–W5 are proposed, not run (Appendix C), and the v0.7.2-line account build OZ-PB signs for has no recorded source.

9 Formal Results

The evaluator is about 2,400 lines of dependency-free `no_std` Rust without `unsafe`, in a subset Aeneas translates: no traits, closures or iterator adaptors, and every loop a `while` over one slice. Charon reads the MIR of rustc nightly-2026-09-17, and Aeneas [12] translates it into a generated 3,653-line Lean file, which rule-core's CI regenerates and diffs; CI pins all 940 exported theorems to Lean's three standard axioms. We say *proved* only of kernel-checked statements, each about one object: the *model*, the *translation*, a two-event *account model* with no Rust counterpart, a tested *host model*, or Perch's canonical encoding. Proving that a translated function returns `ok` on every input rules out panics and divergence.

Definition D1 (contexts and authorizations)**.** A context is $c ::= \mathsf{call}(C, f, \vec{v}) \mid \mathsf{create}(h)$. An authorization is $a = (\vec{c}, \vec{j}, S, \ell, g)$: one entry's contexts in preorder, root first; one rule index per context; the claimed signers; the ledger; and the account's generation. Lean: `Chain.Context`, `Authorization`

Definition D2 (rules and *allows*)**.** A rule r has a scope sc_r (a contract and function list, false on creations), conditions Φ_r, signers Σ_r, a threshold k_r, and optionally a cap (λ_r, P_r), a last ledger u_r and a generation g_r; $\mathit{allows}(r, c, S) \iff sc_r(c) \wedge \bigwedge_{\varphi \in \Phi_r} \varphi(c) \wedge k_r \le |\Sigma_r \cap S|$, counting distinct signers. A rule set allows c under S when one of its rules does; $R' \sqsubseteq R$ when R allows every (c, S) that R' allows. Lean: `Rule.allows`, `RuleSet.narrower`

Table 1: One task in both designs, run in the walkthrough. The task: an app key may send XLM from the account only to the shop, at most 10 XLM in any 100-ledger window. One soroban-sim session per design (soroban-env-host 29.0.2 in soroban-sim, protocol 29, applying with enforcing authorization; simulation's recording-mode host build, not the exact-metering one), so `__check_auth` ran: the OpenZeppelin v0.8.0 account 8286422f… with oz-policy-builder's interpreter 6704996f… and OpenZeppelin's `spending_limit` 65e70aad…, carrying the rule policy-synth synthesises; and Perch f0275f4 with one grant. Signers: a delegated Stellar account on the OpenZeppelin side, as oz-policy-builder requires; an Ed25519 key checked by a verifier contract on the Perch side. Each refusal is an enforcing simulation that left the ledger unchanged.

Step	Request	OpenZeppelin account with oz-policy-builder	Perch
1	The owner installs the rule	Applied	Applied
2	The app pays the shop 6 XLM	Applied	Applied
3	5 XLM more in the same window	Refused by `spending_limit`, #3221 `SpendingLimitExceeded`; the interpreter is not called	Refused, auth error 5; no verifier call
4	1 XLM to another recipient	Refused by the interpreter, #100 `ArgMismatch`, after `spending_limit` recorded the spend (rolled back)	Refused, auth error 5; no verifier call
5	Another key signs in the app's name	Refused by the host's signature check on the app's entry; no policy runs	The evaluator accepts; the verifier traps in the host's signature check
6	A key not on the rule signs as itself	Refused by the account, #3016 `UnauthorizedSigner`	Refused, auth error 5; no verifier call
7	100 ledgers later, 5 XLM	Applied	Applied

Definition D3 (*authorize* and runs)**.** Given one record per rule, ρ, $\mathit{authorize}(R, \rho, a)$ refuses unless $|\vec{\jmath}| = |\vec{c}|$ and $|\rho| = |R|$, and then unless (i) for each i, $r = R_{j_i}$ exists, is unexpired ($\ell \le u_r$), is current ($g_r = g$ when present) and $\mathit{allows}(r, c_i, S)$; and (ii) every $s \in S$ is a signer of a chosen rule. It then (iii) charges, in context order, each context whose chosen rule is capped to that rule's record: a charge refuses a missing or negative amount, a ledger before the newest entry, overflow, a full history, or an in-window total above λ_r; otherwise it evicts entries at or before $\ell \dot{-} P_r$ and appends the spend. It returns the records or the first refusal. A run applies authorizations in order; a refused one leaves the records unchanged.

Lean: `RuleSet.authorize`, `Params.check`, `RuleSet.Accepts`

Appendix B gives D4–D9 and the remaining statements.

T1 (Total equivalence of the evaluator). *On every input the translated evaluator succeeds, never panicking or diverging, and its value denotes the model's result on the denoted inputs; this says nothing about allocation failure or compiled Wasm.*

Hypotheses: None: every input

Evidence: Equiv.authorize_eq · PROVED · TRANSLATION

T2 (Acceptance implies permission). *If evaluation accepts, the selected rule set allows every supplied context for the signer list it was given. The theorem takes that list as an input; authenticating it is the shell's obligation (§6).*

Hypotheses: The signer list is an input; the shell authenticates it (O1)

Evidence: Extracted.authorize_allows_all and Chain.RuleSet.authorize_allowsAll · PROVED · MODEL + TRANSLATION

T6 (Cap-window safety). *From empty records, in any run, counting its accepted authorizations, the charges made to a capped rule with a non-negative limit after $L - P$ and through L sum to at most its limit, for every ledger L; the model also bounds every P consecutive ledgers from ledger 1 (Appendix B).*

Hypotheses: Empty records; a cap with a non-negative limit; for the bound over every P consecutive ledgers, every authorization at ledger 1 or later

Evidence: RuleSet.window_le_limit; Extracted.runAuthorizations_window_le_limit · PROVED · MODEL + TRANSLATION. RuleSet.period_le_limit · PROVED · MODEL

T8 (Grant target containment). *A document lowered with account address A, excluded set X and generation g allows no call to A or to an element of X, and attenuation preserves this. In the account, X is the adopted recovery controller and the zero-knowledge adapter and pool of its recovery configuration (§5).*

Hypotheses: The document is lowered with account address A, excluded set X and generation g; `ArgsRoom`

Evidence: Document.Doc.lowerScoped_allows_excluded; Extracted.lower_scoped_allows_excluded; derives_allows_excluded · PROVED · MODEL + TRANSLATION

T9 (The containment chain). *A grant issued from a valid grant document, then attenuated and accepted, allows only the source document's actions and no call to the account or the excluded set at issue, and a nonempty batch is evaluated at the grant's issuing generation. The account builds X from its storage (O4) and rechecks it in* `__check_auth` *(*`unproven.scoped-admin-change`*).*

Hypotheses: `Doc.Valid`; `ArgsRoom`; a nonempty batch; X is the translated `excluded`

Evidence: Document.Account.chain; Extracted.chain · PROVED · MODEL + TRANSLATION

T10 (Recovery invalidation). *In the account event model, the generation advances by exactly the number of completed recoveries, and a grant issued before one, or derived from such a grant, accepts no nonempty batch at the new generation. Administrative rules are generation-independent.*

Hypotheses: A nonempty batch; the authorization carries the account's current generation (O3)

Evidence: Document.Account.run_generation; issued_stale; chain_after_recovery · PROVED · ACCOUNT MODEL (no Rust counterpart); stale-rule rejection: Extracted.authorize_generation · PROVED · MODEL + TRANSLATION

From the theorems to the account. The account calls the Rust `lower` and `lower_scoped`, whose translations the theorems cover, on install, and `rule_eval::authorize` in `__check_auth`. Perch's integration suite runs rule-core's 57 Lean-generated traces through natively compiled `apply_doc`, `apply_scope` and Perch's `check_auth`, with the real Ed25519 verifier and the generation set from each trace, and checks stored rules, verdicts and spending records against the model. The theorems hold over one fixed rule slice, while the shell builds a slice of the selected rules for each authorization; carrying them through that selection is obligation O2 of Table 11.

Proposition 1 (argued). *Under obligations O1–O8, T1 with* `RuleSet.authorize_chosen`*, the per-rule forms of T8 and T9, and* `authorize_generation` *(T10) give, for each context whose selected rule belongs to a grant: the context is allowed by the grant's source document, calls neither the account nor its excluded set, and is refused after a completed recovery.*

The proposition is argued, not mechanized: O1, O3, O4 and O6–O8 are tested, O2 is argued and tested, and O5, that compiler outputs satisfy `Doc.Valid`, is unproven (`unproven.perch-validation`).

Trust base. The formal results assume the Lean kernel and its three axioms, `propext`, `Quot.sound`, `Classical.choice`; Charon 7b462d9, Aeneas ab34bd2 and Aeneas's Lean library's model of Rust primitives; the MIR of rustc nightly-2026-09-17; the denotations and `Doc.allows`, written from Perch dff2a73; and the host models, tested against soroban-env-host 29.0.1 and 30.0.0. The deployed account also relies on rustc 1.97.1 and its Wasm back end; the document compiler, which turns grant JSON into the model the account lowers; the tested shell, verifiers and recovery controller; and soroban-env-host (Fig. 5).

10 Evaluation

Method. The reference run executes the built account, compiler, Ed25519 verifier and recovery controller as contract Wasm in soroban-sim [25] with soroban-env-host 29.0.2 on a local protocol-29 ledger, applying with enforcing authorization; setup loads cost and configuration entries from a testnet fixture (Appendix A). Recovery uses real guardian signatures; no zero-knowledge or passkey flow runs. After each command the driver fingerprints every written entry's key, XDR and live-until value.

RQ1: enforcement. Perch's integration suite runs rule-core's 57 Lean-generated traces through the natively compiled account and checks stored rules, verdicts and spending records against the model (§9). The reference run then exercises the account as Wasm: each of its 27 commands (Table 3) gave its expected result. Between them they submit 27 transactions: command 1 submits 11 for setup, command 19 submits 2 to upload and deploy the game plus one to install grant 3, and each other applied command one; 11 more are refused in enforcing simulation with the fingerprint unchanged. The final balance is 93 XLM (100 − 6

$-2-1-1+3$) and the grant generation 1: the cap refuses commands 4 and 26, the generation command 13 and the credential index command 15 (Fig. 3).

On 5 November 2026, a scripted run of a preview of Nido, with virtual passkeys, made 11 successful testnet transactions: account creation; an account document naming the passkey verifier `perch-webauthn-verifier@0.1.2`; the game's two-rule grant; two lost guesses; and a one-guardian *Loss* recovery to completion, which moved the grant generation to 1. The wallet, not the chain, declined a third guess that would exceed the cap.

RQ3: cost, Perch only. Including `__check_auth` and verifier calls, installing grant 1 took 8,493,201 CPU instructions, a capped spend 2,752,173 and a recovery completion 14,220,904, against a per-transaction limit of 400M; a lost-key completion at the document caps, measured in process, uses 40.7% of it. On the testnet preview, an account holding the game's grant after two capped transfers stores 3,308 bytes in 8 entries (1.677 XLM a year in rent), and a passkey-signed transfer through a grant rule declared 5,944,258 instructions and paid 12,889 stroops. These setups differ, so the figures are not comparable with one another.

Threats to validity. The host is simulation's recording-mode build, not the exact-metering one, so counts can differ from stellar-core's metering, and memory and I/O are not reported. Testnet resets periodically, and the testnet transactions (Table 5) are not yet archived with their envelopes. The testnet game ran from a development branch with an uncommitted patch. The comparator was run on one task. Local runs use host 29.0.2, which the host models were not tested against.

11 Related Work

Capabilities and delegation. A grant addresses a form of the confused-deputy problem [11]; the deputies are the account's callees and the owner's `execute`, which runs with the account as invoker. SPKI/SDSI [8] narrows authority by intersecting tags, and macaroons [1], Biscuit [2] and UCAN [27] attenuate bearer tokens offline with caveats. Perch's grants are state installed in the account, not tokens a holder carries. A windowed cap needs past spending, which a bearer token cannot carry without an external store; in Schneider's terms [24] a cap makes the evaluator a stateful execution monitor, while an OZ-PB `read` makes a decision depend on external mutable state, outside his history-only setting.

EVM account abstraction. ERC-4337 [3] accounts validate in `validateUserOp`; ERC-7579 [33] validators and hooks are the EVM counterpart of OpenZeppelin policies; ERC-7710 [17] redeems delegations, and ERC-7715 [13] lets an application request permissions from a wallet. Zodiac Roles [9] scopes a role to targets, functions and parameter conditions with refilling allowances, close to Perch's targets, conditions and caps. Perch charges caps in the account's own decision; OpenZeppelin accounts charge them in a policy contract called from `__check_auth`.

Policy languages and synthesis. Cedar [6] pairs a policy language with formal models, and links a Lean model to its Rust implementation by differential testing [7]; Perch instead proves a functional translation of its Rust evaluator equivalent to a model, which still leaves the compiled code trusted. All three systems deny by default: Cedar adds `forbid`, Perch has no negation, and oz-policy-builder has `ne` and `nin`, which deny when an operand does not resolve. OZ-PB's synthesis from a recorded transaction relates to policy mining [32].

Soroban verification. Certora Sunbeam [5] checks specifications against compiled Wasm, and Komet [22] reasons about Wasm with K semantics; Perch's Komet specifications check its earlier `perch_program::rpn::eval`, not `rule_eval`. Both complement a proof of a pure evaluator.

12 Limitations

Proofs, shell and host. The theorems concern the translation, not the Wasm that rustc 1.97.1 builds (Fig. 5); the specification is trusted, including `Doc.allows`, written from Perch's earlier account (dff2a73), not from the pinned gate. The shell obligations of Table 11 are tested or argued, not proved, and no single theorem covers the whole authorization path, including mixed administrative and grant batches. The host models omit TTL, archival, fees, footprints and pre-host validation.

Grantees (A1) and submitters (A2). A key named by several live rules has the union of their authority; caps are per rule, not per grant. Each context is checked alone, so a rule meant for a call the game makes also allows that call made directly. Allowances used by `transfer_from`, and funds held by other contracts, move without the account's authorization, and a token rule that does not restrict functions also allows `approve`. An owner-authorized `execute` runs as the account, so no rule or cap sees the callee's `require_auth`; an administrative rule that allows it has unlimited authority over its targets. Replacing a capped rule or its cap starts an empty window. Delegated signers do not bind rule IDs; a rule edited in place between signing and inclusion decides under its new contents; a refused entry can be resubmitted until it expires; and `apply_grant` takes no expected version, so two pre-signed installs for one grant ID can land in either order.

Owner-key theft and recovery (A3, A4). Under Loss a thief holding the owner key can cancel every recovery; Protected without an enrolled baseline protects no better, and a full guardian quorum is trusted. The owner may schedule an upgrade outside an attempt and execute it after 120,960 ledgers (seven days at the nominal five-second ledger); new code inherits the account's storage, including the revoked set and generation, and the proofs do not cover it. A completion marker is consumed by the target's canonical identity, not its exact bytes (issue #15).

Trusted components (A5). The account trusts its verifiers and its recovery controller, which gets no registry provenance check. A verified verifier name means only that the registry deployed that code at its own hash; its manager decides which hashes it deploys, its admin can replace the registry's code, and one key holds both roles on testnet and mainnet. WebAuthn origin is not checked.

Resources and audit. Bounded records and arithmetic can refuse permitted actions. Document caps (8 signers, 10 rules, 8,192 canonical bytes) are meant to keep a thief holding the owner key from making recovery too expensive to complete; disk-read bytes are not measured. `renew` does not extend installed-grant records. No external audit has been completed.

13 Future Work

Everything here is proposed, not implemented. We propose an agent runtime with the same split, a pure engine that decides and a host that mediates, in which a root agent holds a Perch grant and derives child grants with delegation-tree budgets (T7 is their model result). We also propose exact entry roots (Perch #69 and rule-core #17), an entry point for attenuated grants, a recording front end that emits grant documents, a Lean lemma for O2, and exact-metered workloads W1–W5 on both designs.

A proved bridge from Cedar. Proposed, not under way. The Lean library of cedar-spec [4] loads unchanged into rule-core's proof toolchain; both semantics are total definitional interpreters, so a simulation between them is one commuting equation. In an unpublished experiment we proved it for a cut-down fragment, with no `sorry`, and composed Cedar's `allowed_iff_explicitly_permitted_and_not_denied`, the simulation and rule-core's `allows_eq` down to the extracted Rust. Perch would borrow Cedar's specification, not its large proofs, through a Lean-verified compiler from a Cedar fragment to rule-core rules, each installed rule set checked in the kernel. Known limits: Lean-version lockstep between cedar-spec and Aeneas, Cedar's 64-bit integers against `i128`, and a parser that is tested, not proved.

14 Conclusion

Perch decides each authorization with a pure evaluator linked into the account, and a completed recovery stops every earlier grant from authorizing by advancing one counter. The evaluator is proved in Lean over its translation and recovery invalidation in an account model; the shell, binary and host are tested or trusted. Against oz-policy-builder, the designs differ in where a limit over time lives, how a rule is written, and the kind of evidence behind each. What remains is to merge the stack, complete an external audit, and measure both designs with exact metering.

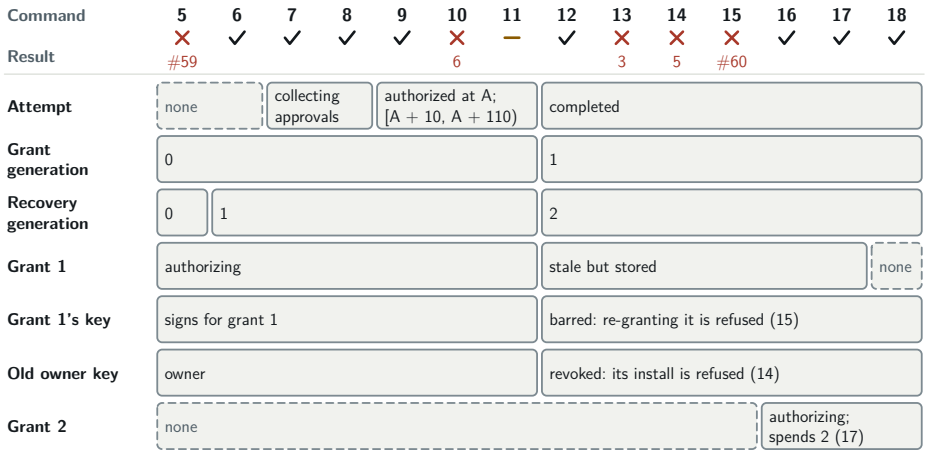
Table 2: Status on 7 November 2026. *Deployed*: recorded in Perch's deployment manifests and present on the network. *Preview*: a separate testnet deployment outside those manifests. *Merged*: on a main branch, not deployed. *Open PR*: unmerged code. *Proposed*: not implemented. *Not done*: not started or not complete.

Component	Status	Where	Evidence
Perch 0.5.1 (earlier design: an OpenZeppelin account-library fork with two Perch policy contracts), document compiler, recovery controller, Ed25519 and WebAuthn verifiers, ZK pool and adapter	DEPLOYED	Perch `c135aca`: testnet 2026-10-22, mainnet 2026-10-26	`deployments/testnet.json`, `mainnet.json`
This paper's account (deciding with `rule_eval`; installed grants; credential index; named verifiers; rebuilt recovery hooks)	OPEN PR	Perch f0275f4, head of the open stack ending at #68	Unmerged
The same contract source on testnet	PREVIEW	Separate deployment, 2026-11-05, ledger 5,577,521; account Wasm 0bdbdd14… (106,584 B)	Built from a410123, whose contract sources equal the pin's
Wallet on that deployment (Nido)	PREVIEW	nidohq/nido#356 (draft)	11 testnet transactions (§10)
rule-core evaluator, model and proofs	MERGED	rule-core cbabe59	CI regenerates the translation, builds the proofs and checks the claims ledger
Exact authorization-entry roots	OPEN PR	Perch #69 and rule-core #17	No theorem at cbabe59
External audit of the Perch contracts (account, compiler, recovery controller, verifiers, ZK pool and adapter, registry) and of rule-core's evaluator	NOT DONE	—	Only the vendored UltraHonk base verifier, from Nethermind, has one (by OpenZeppelin); Perch's zero-knowledge additions have none
Contract exports under this paper's names	PROPOSED	Table 4	Exports unchanged
Agent runtime; x402 payments; Cedar authoring; delegation-tree budget runtime; account entry point for attenuated grants; in-page proof check	PROPOSED	§13	Not implemented

A Artifacts and Reproducibility

Sources. This paper describes Perch at revision `f0275f41c5ab47c903bbfe9986b4bf3239c01b11` [31] and rule-core at `cbabe594ae2df408991e825e56ddd618ea29541b` [30]. The reference run uses soroban-sim at `d02938430f0ac41f34e5e3e4c8c48d85b871bd68` and the Guess the Number fixture at `df80831ac6bc2a2110cab15654726456b53cc8f3` [10].

Proofs. The proofs reproduce from rule-core with Charon 7b462d9, Aeneas ab34bd2 and Lean v4.35.0: `tools/extract.sh --check` regenerates the translation and fails on any difference, and `lake build` in `formal/` checks the theorems; rule-core's README gives the full commands. Perch's canonical-encoding pack-



5 Collect grant 1: refused, ScopeLive (#59)
6 Enroll recovery: Loss, two guardians
7 Begin an attempt
8 Guardian 1 approves
9 Guardian 2 approves: authorized at ledger A
10 Complete: refused, NotExecutableYet (6, controller)
11 Advance 11 ledgers: no transaction
12 Complete: applied in one transaction
13 Spend 1 with grant 1: refused, StaleScope (3)
14 Old owner installs: refused, RuleEvalRefused (5)
15 Re-grant wiped key: refused, WipedCredential (#60)
16 Install a fresh credential: grant 2
17 Spend 2 with grant 2: balance 94 → 92
18 Collect grant 1: anyone may; index kept

Fig. 3: Recovery in the reference run, commands 5–18. For each command 5–18, its result and the state it changes. Completion (command 12) moves the grant generation from 0 to 1 and revokes the replaced owner key. Grant 1 stops authorizing there but stays stored until anyone collects it (command 18); its key stays barred. Command numbers match Table 3.

age uses Lean v4.33.0. rule-core's CI also checks a ledger of 180 claims (107 proven, 40 tested, 19 assumed, 14 unproven).

Reference run. Setup loads protocol-29 cost and configuration entries from a testnet fixture (ledger 5,301,502; testnet now runs protocol 30) and places the compiler and the Perch 0.5.1 policy contracts at the addresses the account derives from its pins; a 562-byte registry stub resolves the verifier name `ed25519@1.0.0` offline, so the production registry's governance is not tested. Refusals caught in enforcing simulation are not submitted, so an unchanged fingerprint says nothing about fees for failed, submitted transactions. A native program and a WASI reactor run the same driver over the same pinned bytes and agree on every returned field except wall-clock time. The HTML edition embeds a source bundle with the Rust JSON-command driver, its native entry point, a 5-function WASI shim, the browser worker, dependency locks, the contract build script, the reference command sequence and a comparison checker; `projects/perch-browser/README.md` in the bundle gives the pinned build commands. Like Perch's own release build, the build rewrites the account's infrastructure pin files with the hashes of the contracts built in the same run, so the embedded account (3038f5dc…) differs

Table 3: The reference run. Commands are numbered as in the live experiment. CPU is the host CPU instructions of each applied, signed transaction in soroban-sim with soroban-env-host 29.0.2, applying with enforcing authorization, so `__check_auth` and verifier calls are included. The host is simulation's recording-mode build, not the exact-metering one, so counts can differ from stellar-core's metering. Refusals are caught in enforcing simulation and never submitted. Auth errors 3 and 5 are `__check_auth`'s codes inside `Error(Auth, InvalidAction)`; #59 and #60 are account entry-point errors.

#	Command	Expected result	CPU instructions
1	Create / reset	11 setup transactions; balance 100 XLM; generation 0	87,218,534; 91,002,996; 16,141,851; 12,030,445; 2,252,729; 55,758,065; 92,237; 1,760,165; 185,494; 1,508,981; 915,820
2	Install grant 1	Applied	8,493,201
3	Spend 6	Applied; 100 → 94	2,752,173
4	Spend 5	Refused, auth error 5 (cap); fingerprint unchanged	—
5	Collect grant 1	Refused, `ScopeLive` (#59)	—
6–9	Enroll; begin; guardian 1; guardian 2	Applied; authorized at ledger A	13,733,435; 13,670,993; 1,719,379; 2,595,288
10	Complete	Refused, controller `NotExecutableYet` (6)	—
11	Advance 11 ledgers	View only	—
12	Complete	Applied; generation 0 → 1	14,220,904
13	Spend 1 with grant 1	Refused, auth error 3 (`StaleScope`)	—
14–15	Old owner installs; re-grant the wiped key	Refused, auth error 5; `WipedCredential` (#60)	—
16–18	Install grant 2; spend 2 with grant 2; collect grant 1	Applied; 94 → 92	8,674,696; 2,815,558; 1,365,038
19	Install grant 3 (the game)	Applied	3,663,659; 1,006,385; 10,697,803
20–23	Game with missing or wrong authorization, either outcome	Refused (host authorization error, or auth error 5 for a wrong credential)	—
24–25	Guess 0, twice	Applied; 92 → 91 → 90	3,992,899; 4,104,949
26	Guess 0, a third time	Refused, auth error 5 (2-XLM cap)	—
27	Winning fixture	Applied; 90 → 93; cap record unchanged	3,082,877

from the testnet preview's (0bdbdd14…). The account still pins the Perch 0.5.1 policy contracts and reports them through `infra()`, but never calls them. The demo's driver maps the names this paper uses to the pinned exports of Table 4 at its call boundary. An executable walkthrough of the task in Table 1, in both designs, accompanies the HTML edition.

This paper. `python3 docs/perch-paper/build/paper.py` renders both editions, the HTML edition and this PDF, from shared sources, and `docs/perch-paper/latex/build.sh` builds this PDF with a pinned Tectonic.

Table 4: Account operations. The first column is the name this paper uses; the second is the symbol exported at the pin and by the testnet preview. Exporting the first column's names is proposed, not done. *Owner authorization* means an account-document rule that targets the account and whose function policy allows the entry point; a rule with no function list, like the constructor's, allows all of them. `docs/account-interface.md` lists the account's other entry points.

Name in this paper	Pinned export	Inputs → result	Authority and effect
`apply_account_doc`	`apply_doc`	Document bytes, approval-valid-until ledger, optional expected revision → document hash	Owner authorization, or the recovery-completion path (§06); installs account policy.
`apply_grant`	`apply_scope`	Grant ID (`u32`), document bytes → document hash	Owner authorization; installs or replaces a grant stamped with the current generation.
`remove_grant`	`remove_scope`	Grant ID → unit	Owner authorization; removes the grant's rules, tags and spending records.
`gc_grant`	`gc_scope`	Grant ID → unit	Anyone, only when the grant's generation is older than the account's; otherwise `ScopeLive`.
`grant` / `grants`	`scope` / `scopes`	ID → optional grant view; start and count → IDs	Read-only; the list includes stale grants not yet collected.
`grant_generation`	`scope_generation`	None → generation	Read-only; only a recovery completion advances it.

Table 5: Deployed artifacts this paper cites. Testnet is reset periodically. Manifests: `deployments/testnet.json`, `mainnet.json` at `f0275f4`. The embedded build's artifacts and their hashes are listed with the source revisions in the reproducibility material.

Artifact	Network	Identifier
Perch 0.5.1 account (`c135aca`), 70,702 B	testnet, mainnet	Wasm `2c976bfd98678f3b6fcf5514420a81ef5e5b9fc1c5b6c60ec6f2dd64448479d5`
Perch 0.5.1 policy contracts, 17,999 B and 12,923 B	testnet, mainnet	Wasm `f884913eba4a9491f8bcf89d1a687381c7c0699b148f98f8dc344e9a6223b26e`; `8c043dcc24f944d7c3e349fb5fd83cedba4a9124b5791b625bd76b520b0f4a80`
This paper's account, preview deployment, 106,584 B	testnet	Wasm `0bdbdd142c953976056196cc837f84667e0fb64e731b007cc45d8b852c4ce844`
Nido preview account; completion transaction	testnet	`CBOUTD6XXCUB4QTIT4T5D4U5EMAUTKYAKPSKBTZF4RXGD3OWQTXINLZM`; `161cd96849732ba17740c05157d60141e765099fe854929101113e48427917ba`
Storage-census account (fresh preview account holding the game grant)	testnet	`CDAPPQ5YKFK2GBAP6TSKPHHHDNDZWBYFF4AVC5I4BVCDJ5MK6CPFFX3V`
Smoke-test account; 1-stroop passkey transfer through a grant rule	testnet	`CBJ3PMMMQOQGGHYPXRNS4S4LOIWFPXVE2HOLY7DRNWK6PPE4EHVLCDXE`; `3e652c739cd265f5cd9d9434c2f5f2c927c66dd810f7cf2efe55b77f45a0234c`

Table 6: Ending authority. Invalidation is a policy event, collection a storage operation, and archival neither. Only a completed recovery moves the grant generation; otherwise the owner revokes grants one at a time.

Mechanism	Triggered by	Stops authorizing	Removes storage	Bars the credential later
Expiry (`not-after-ledger` N: the rule authorizes through ledger N − 1)	The ledger reaching N	That rule	No	No
`remove_grant`	Owner	That grant's rules	Rules, tags, spending records	No
Completed recovery	The controller's approved completion	Every earlier grant (generation + 1); replaced and removed administrative credentials	No	Yes: grant credentials through the credential index, administrative credentials through the revoked set
`gc_grant`	Anyone, only for a grant whose generation is older; an expired but current grant is not collectable	Nothing new	Rules, tags, spending records; the credential index is kept	Unchanged
Archival (TTL lapse)	Rent not paid	Nothing; the entry is unavailable, never absent, until restored	No	No

B Formal Appendix

B.1 Further definitions

Definition D4 (window and *spent*)**.** For period P, a charge at ledger t is in the window at L when $L \dot{-} P < t \leq L$. $spent_j(R, \rho, \vec{a})$ lists the ledger and amount of every positive charge the accepted authorizations of run $\vec{a}$ make to rule j.

Lean: `Params.inWindow`, `RuleSet.spent`

Definition D5 (narrowing and derivation)**.** Narrowing $R' \sqsubseteq R$ is as in D2. $Derives(R_0, R)$ holds when R is reached from R_0 by zero or more successful attenuation steps (Table 9). $TraceNarrower(R', R, f)$ holds when every run that R' accepts from empty records, R accepts from its own empty records with each choice j renumbered to $f(j)$; a step that drops rule i renumbers every later index down by one. Lean: `RuleSet.Derives`, `RuleSet.TraceNarrower`

Definition D6 (documents and lowering)**.** For a document d on account A, $Doc.allows(d, A, c, S)$ holds when some rule of d covers c's contract and function, its argument constraints hold and its principals are signed by S; no rule allows a creation. $Doc.Valid(d)$ holds when every rule that requires all its signers names distinct signers, fewer than 2^{32} of them, and no rule has a not-after ledger of 0. $lower(d, A)$ is the rules of an account document, followed by the recovery rule when d configures recovery. With $X = excluded(d_{\text{admin}}, A)$, which is A,

Table 7: Adversaries and what holds against each. Trusted, not adversaries: the Soroban host and validators; the verifier contracts documents name; the adopted recovery controller; the document compiler, which turns grant JSON into the model the account lowers; rustc 1.97.1, which builds the deployed Wasm; the wallet's display of what is signed; and, for the proofs, the Lean kernel, Charon, Aeneas and the specification. T-labels refer to the statements in §9, O-labels to the shell obligations there.

Adversary	Can do	What Perch claims (evidence)	Residual
A1 Grantee (app or agent key)	Sign within rules that name its key; build any invocation tree; cite any rule that names its key	An accepted authorization is allowed by the rules it cites (proved for the evaluator, T2; signer authentication, selection and context conversion tested, O1, O2, O7). No grant rule allows a call to the account, its adopted recovery controller, or the ZK adapter and pool of its recovery configuration (proved for lowering under `Doc.Valid`, T8–T9; O5 unproven; runtime recheck tested). Each capped rule's window stays within its limit (proved for the evaluator from empty records, T6; selection and record persistence tested, O2, O6)	A key named by several live rules has the union of their authority. Caps are per rule, not per grant. Each context is checked alone, so a rule meant for a call the game makes also allows that call made directly (Figure 6c). Allowances, and funds held by other contracts, move without the account's authorization
A2 Submitter or relayer	Wrap signed entries, choose timing and fee, resubmit a refused entry until its expiration ledger	An external signer's signature binds this account and the selected rule IDs (tested)	Delegated signers do not bind rule IDs. A rule edited in place between signing and inclusion decides under its new contents
A3 Thief of the owner key	Everything the owner can do	Under Protected with an enrolled baseline, once the recovery condition authorizes an attempt, the account authorizes only its completion; changing recovery or upgrading needs the condition's evidence (tested)	Without a baseline, Protected is no stronger than Loss, under which the thief can cancel any recovery. Allowances survive recovery. An owner upgrade executes after 120,960 ledgers
A4 Guardians below quorum	Approve attempts	Completion needs the quorum, the delay and the window (tested)	A full quorum is trusted
A5 Manager or admin of `perch/verifiers`	Admit verifier names; the admin can upgrade the registry	An applied account, and its pinned infrastructure addresses, are unaffected by later registry actions (argued)	Names resolved by later installs follow the manager. One key holds both roles on testnet and mainnet

the adopted controller and, if configured, the zero-knowledge adapter and pool, $\mathit{lowerScoped}(d, A, X, g)$ refuses a document that configures recovery or has a rule whose target is A, a member of X or the account's self-administration, and otherwise lowers d's rules stamped with generation g. rule-core's conditions, `ArgEq`, `ArgIn`, `ArgStrPrefix`, `ArgI128AtMost` (Table 9), are an equality, a set membership, a string prefix and a signed upper bound. `is-self` and `address-eq` lower to an

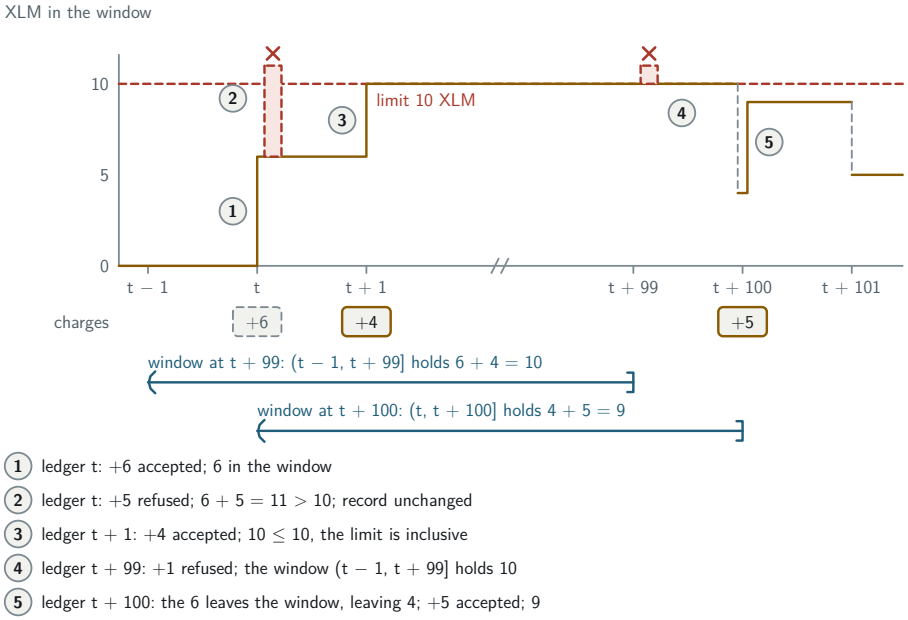


Fig. 4: Grant 1's cap, 10 XLM per 100 ledgers, on illustrative ledgers. At ledger L the window holds charges made after $L - 100$ and through L. Borders show the window at $t + 100$: solid inside it, dashed outside. A refused transfer leaves the spending record unchanged. A charge stops counting exactly 100 ledgers after it was made.

address equality, `u32-eq` to an equality on the argument's encoding, `string-in` to a membership and `string-prefix` to a prefix; a cap adds the upper bound of the largest `i128` on argument 2, which is a type guard.

Lean: `Doc.allows`, `Doc.Valid`, `Doc.lower`, `Doc.lowerScoped`

Definition D7 (account events)**.** An account state is $s = (A, d_{\text{admin}}, g)$. The event applyDoc$(d)$ sets $d_{\text{admin}} := d$; the event completeRecovery(t) sets $d_{\text{admin}} := t$ and $g := g + 1$, and is refused when g is the largest `u64`. A run applies events in order, a refused one leaving s unchanged. A grant document d is *issued* in s as $\mathit{lowerScoped}(d, A, \mathit{excluded}(d_{\text{admin}}, A), g)$.

Lean: `Account.step`, `Account.run`, `Account.issue`

Definition D8 (host run)**.** A model of soroban-env-host running a transaction of contract calls under enforcing authorization. Every call is a frame that a failure rolls back. `require_auth` of an address holds when the frame's direct caller is that address (invoker authority), or when an authorization entry of the address matches; on its first match the address's `__check_auth` runs on the entry's contexts in preorder, then the expiration is checked and the nonce consumed. Stellar-account signers, `require_auth` inside `__check_auth`, `require_auth_for_args`, `authorize_as_current_contract`, contract creation as a call and source-

account credentials are outside the model.

Lean: `Soroban.Tx.run`

Definition D9 (denotation)**.** $[\![\cdot]\!]$ maps the translation's values (byte vectors, `u32` and `u64` scalars, rules, caps, records, contexts, signers and outcomes) to the model's. T1 and T12 are stated through it: for every input x there is a result y with $\mathit{authorize}_{\mathrm{T}}(x) = \mathsf{ok}(y)$ and $[\![y]\!] = \mathit{authorize}([\![x]\!])$.

Lean: `Equiv.denoteRules`, `Equiv.denoteAuthorized`

Ending authority. A grant is *invalidated* when the account's generation passes the generation it was stamped with; it stays stored and refuses every nonempty batch. *Removal* (`remove_grant`, by the owner) deletes a grant's rules, tags and spending records. *Collection* (`gc_grant`, by anyone) deletes the storage of an invalidated grant and keeps the credential index. *Archival* is a lapse of an entry's time to live: the entry is unavailable until restored, not absent.

The period window at ledger 0. The second form of T6 assumes every authorization is at ledger 1 or later. At ledger 0 the eviction cutoff $0 \mathbin{\dot{-}} P$ saturates to 0 and evicts entries at ledger 0, so each spend at ledger 0 is checked alone: rule-core's `Cap.period_ledger_zero` records two spends of the whole limit at ledger 0, and the P ledgers from 0 then hold twice the limit. Stellar's ledgers start at 1.

B.2 Canonical identity and signature binding

CANON v1. `doc_hash` is the SHA-256 of a document's CANON v1 bytes, a restriction of RFC 8785 [21]. The CANON version is not part of the hashed bytes, so a `doc_hash` is read under the identity the account reports (`capabilities().doc_identity`, which is `canon_v1`). T13 proves the encoding injective on the modeled document type; `configPreimage_ne_emitDoc`, `rulePreimage_ne_configPreimage` and `rulePreimage_ne_emitDoc` prove that configuration, rule and document hash preimages never coincide.

Digests. With $H = \mathrm{SHA\text{-}256}$, the account's 32-byte ID id_A, the tag $\tau =$ `perch/account/auth-addr-v1` and the selected rule IDs ρ:

$$P = H\big(\mathrm{XDR}(\texttt{HashIdPreimage} :: \texttt{SorobanAuthorization}\{\mathit{nid}, \mathit{nonce}, \mathit{exp}, \mathit{inv}\})\big),$$
$$B = H(\tau \,\|\, id_A \,\|\, P), \qquad D = H\big(B \,\|\, \mathrm{XDR}(\texttt{Vec}\langle\texttt{u32}\rangle\ \rho)\big).$$

P is the payload for a legacy `Address` credential; `AddressV2` and delegated entries hash `SorobanAuthorizationWithAddress`, which adds A. An external signer signs D; a delegated signer approves the host's address-bound payload, which omits ρ. OpenZeppelin v0.8.0 has an external signer sign $H(\mathrm{XDR}(\texttt{AuthDigestPreimage}\{A, P, \rho\}))$ and a delegated signer authorize the preimage through `require_auth_for_args`. In its v0.7.2 line, an external signer signs $H(P \,\|\, \mathrm{XDR}(\rho))$, without the account. The following is a paper proof, not mechanized.

Table 8: What a signature binds. The first two rows come from the host's signature payload. OpenZeppelin columns are read from `stellar-contracts` v0.8.0 and v0.7.2 source; the 0.7.2-line build that oz-policy-builder signs for (91a2cd56…) has no recorded source, and its digest is taken from oz-policy-builder's documentation. [1]With a legacy `Address` credential; CAP-0071's address-bound credentials add the account to the host's payload. In oz-policy-builder's deployment, external signers are only Prime seats in rule 0, each accepting only its own account's ID as key data, and the interpreter refuses external signers in rules it polices (error 212); read in source, not run.

Bound to	Perch external	Perch delegated	OZ v0.8.0	0.7.2-line external	0.7.2-line delegated
Network, nonce, signature-expiration ledger	yes	yes	yes	yes	yes
Calls in this entry's invocation tree	yes	yes	yes	yes	yes
This account	yes	yes	yes	**no**[1]	yes
Selected rule IDs	yes	**no**	yes	yes	yes
Rule contents; account revision	no	no	no	no	no
Other entries in the transaction; fee payer	no	no	no	no	no

Lemma 1 (binding). *Suppose the verifier named for an external signer correctly verifies an EUF-CMA-secure scheme and H is collision resistant. An adversary given signatures on $D(t_1), \ldots, D(t_q)$ for tuples $t_i = (A, nid, nonce, exp, inv, \rho)$ of its choice outputs $t \notin \{t_i\}$ with a signature accepted on $D(t)$ only with negligible probability.*

Proof. The map from the tuple to D composes injective encodings with H. The XDR preimage of P is self-delimiting; τ is a constant, and id_A and P have fixed lengths, so $(A, P) \mapsto \tau \| id_A \| P$ is injective; B has a fixed length and the XDR of ρ is self-delimiting, so $(B, \rho) \mapsto B \| \mathrm{XDR}(\rho)$ is injective. If $D(t) = D(t_i)$, the injective encodings give a collision of H; otherwise the output is a forgery. For WebAuthn, the signed message carries D as the client-data challenge. □

Proposition 2 (non-binding). *D does not depend on the contents of the rules ρ names, on the account's revision, on the other entries of the transaction or on its fee payer. For a delegated signer, the payload the host has it approve depends on the account but not on ρ.*

This follows from the preimages: none of these values enters P, B or D, and the delegated payload is the host's address-bound preimage.

Addresses. A Soroban contract ID commits to its deployer and salt, not to its code. The claim that an account calls fixed infrastructure code therefore assumes that the registry deploys each pinned Wasm hash at the salt derived from it and that the deployed infrastructure contracts cannot replace their code; once such code is deployed on a network, no registry action changes it.

B.3 Cryptographic assumptions, verifiers and domain tags

Schemes and verifiers. Ed25519 [14] signatures are checked with strict verification. P-256 signatures are ECDSA with low S enforced; the WebAuthn veri-

fier [29] checks neither the relying-party ID nor the origin. ML-DSA-65 is pure FIPS 204 [18] with a fixed context string, the key being the hash of the public key. The compiler refuses a key not in its verifier's canonical form, so a threshold, which counts distinct identities, cannot count one key twice. The deployed Ed25519 and WebAuthn verifiers trap on a bad signature (`Error(Crypto, InvalidInput)` for Ed25519) rather than return false; the built ML-DSA-65 verifier returns false for a well-formed failing signature. A grant may name a verifier by address alone; each such verifier may supply at most one signer per rule. Ed25519, P-256 and BN254 are not post-quantum. Administrative signers, guardians and the zero-knowledge factor can each change the account's rules, so an ML-DSA-65 signer resists a quantum adversary only if every such path does.

Zero-knowledge factor. It proves knowledge of a secret enrolled as a Poseidon2 commitment in a depth-32 pool, with an UltraHonk proof with KZG commitments over BN254 against a fixed setup point and Fiat–Shamir with Keccak; `docs/zk/` does not name the setup ceremony. The proof's public inputs include Poseidon2 over the account, the enrollment ID and the statement digest, so in combined mode both factors approve one statement. The commitment is public, so the secret must be a uniform field element with at least 128 bits of entropy. The nullifier is fixed by the account, enrollment ID and secret; a completion spends it, after which every proof from that secret is refused. Perch's zero-knowledge verifier has not been audited.

Domain tags. Perch separates its hash domains with the tags `perch/account/auth-addr-v1` (the account-bound payload B, scheme `addr_oz_v1`), `perch/rule` (one rule's bytes), `perch/recovery/statement`, `perch/recovery/config`, `perch/recovery/credential`, `perch/recovery/replacements` (recovery statements, configurations, credentials and replacement sets), and, as Poseidon2 constants, `perch/recovery/zk/v2/leaf`, `perch/recovery/zk/v2/bind`, `perch/recovery/zk/v2/nullifier`, `perch/recovery/zk/v2/auth`. CANON document hashes carry no tag. For comparison, oz-policy-builder uses `prime-seat-grant` (session grants) and `prime.execution.adapter.v5` (the execution adapter's address salt).

Recovery statement. A lost-key or compromise statement is 278 bytes: a 24-byte domain tag, a version byte and an action byte; the network ID and the account and controller contract IDs (32 bytes each); the configuration epoch (8 bytes) and configuration hash (32 bytes); the delay, expiry and valid-until ledgers (4 bytes each); and, for a lost-key or compromise attempt, the attempt ID (8 bytes) and the hashes of the source document, the target document and the replacement set (32 bytes each). Other actions share the first 174 bytes and carry other subjects. Its digest is the SHA-256 of these bytes; the action byte precedes the subject and selects its layout, so the encoding is injective without length prefixes.

B.4 Statements in the appendix

T3 (Signer accounting). *Acceptance implies every supplied signer occurs in a chosen rule, and a threshold counts distinct listed identities. Adding identities*

preserves a rule's permission, though not whole-batch acceptance if an added identity is in no chosen rule.

Hypotheses: As T2

Evidence: Extracted.authorize_signers; Chain.Rule.signed_iff; RuleSet.allows_of_subset · PROVED · MODEL + TRANSLATION

T4 (Attenuation narrows authority). *Successful attenuation by the ten attenuation steps yields a rule set narrower than its parent: under any signers, every action the child allows, the parent allows. Repeated derivation preserves this relationship.*

Hypotheses: Each derivation step is one of the ten attenuation steps and succeeds

Evidence: Chain.RuleSet.Derives.narrower; Extracted.Derives.narrower · PROVED · MODEL + TRANSLATION

T5 (Trace inclusion). *For a single successful derivation lineage, every accepted child history from empty records is accepted by the parent from empty records, with rule choices renumbered past dropped rules. Sibling histories are not covered (§5).*

Hypotheses: Empty records; one derivation lineage

Evidence: Chain.RuleSet.attenuateAll_traceNarrower; Extracted.DerivesAlong.traceNarrower · PROVED · MODEL + TRANSLATION

T7 (Delegation-tree budgets). *From a valid tree with nothing spent, and for its budget operations, spending beneath every node is at most that node's allowance at the end of the run, including when child allowances oversubscribe their parent. Allowances are lifetime totals, not windows, and refills raise them.*

Hypotheses: A valid tree with nothing spent

Evidence: Budget.Tree.spentUnder_le_allowance; Extracted.runBudget_spentUnder_le_allowance · PROVED · MODEL + TRANSLATION

T11 (Authorization-tree coverage). *In the host model, every* `require_auth` *that an account's authorization entry satisfies is for a call that every rule set the modeled* `__check_auth` *checks allows, and* `__check_auth` *receives the entry's invocation tree in preorder, root first, so never an empty batch. The statement covers authorization by entry, not invoker authority, so an owner's* `execute` *falls outside it by construction. The modeled account is an abstract program checking the stateless* `allows`*, not Perch's shell. The model was tested against soroban-env-host 29.0.1 and 30.0.0 on 95 traces it generated (35 named, 60 random) and on 39 traces transcribed from 27 of 74 tests in five host test files, agreeing on every case it expresses (155 of 213 cases). It models CAP-0071 delegation between contract accounts; source-account credentials, classic Stellar-account signers and delegates,* `require_auth_for_args`*,* `authorize_as_current_contract`*, contract creation as a call,* `require_auth` *inside* `__check_auth`*, recording authorization, TTL and footprints are outside it, and the simulator host 29.0.2 was not tested.*

Hypotheses: Contract accounts; `require_auth` satisfied by an authorization entry

Evidence: Soroban.Tx.run_authorized_allowed; run_checkAuth_contexts · PROVED · HOST MODEL (model tested against the host)

T12 (Lowering equivalence). *Translated* `lower` *returns the model's lowering when the document leaves room in its bounded vectors (*`LowerRoom`*), and translated* `lower_scoped` *does, refusals included, under the weaker* `ArgsRoom`*. For a valid document the lowered rules allow exactly what the document allows plus, when recovery is configured, every* `apply_account_doc` *call (*`apply_doc` *at the pin) through the recovery rule, which the shell refuses outside a completion (tested; O8, §6).*

Hypotheses: `LowerRoom` for `lower`, `ArgsRoom` for `lower_scoped`; `Doc.Valid` for the semantic part

Evidence: Equiv.lower_eq; lower_scoped_eq · PROVED · TRANSLATION; Extracted.lower_allows; lower_authorize · PROVED · MODEL + TRANSLATION

T13 (Canonical identity). *Perch's canonical encoding is injective on the modeled document type, so hash identity holds up to SHA-256 collisions. The theorem is about a Lean emitter written to match Perch's Rust* `canon.rs` *and linked to it by golden fixtures and fuzzing, not extraction; JSON parsing and compiler correctness need separate assurance, and no theorem connects this encoding to rule-core's document type. The CANON version is not part of the hashed bytes, so a* `doc_hash` *is read under the identity the account reports (*`capabilities().doc_identity`*); three domain separations are proven:* `configPreimage_ne_emitDoc`*,* `rulePreimage_ne_configPreimage` *and* `rulePreimage_ne_emitDoc` *(rule, recovery-configuration and document preimages are pairwise distinct).*

Hypotheses: The modeled document type; SHA-256 collision resistance

Evidence: PerchFormal.Canon.emitDoc_injective; canonical_bytes_identity · PROVED · PERCH CANON PACKAGE

B.5 The claims ledger

rule-core's ledger records 14 unproven claims, `conformance.encoding`, `unproven.narrower-converse`, `unproven.openzeppelin-signer-mapping`, `unproven.openzeppelin-spending-limit`, `unproven.openzeppelin-authorization`, `unproven.openzeppelin-spending-limit-metering`, `unproven.openzeppelin-expiry-and-signers`, `unproven.generation-counter`, `unproven.perch-meaning`, `unproven.perch-validation`, `unproven.lowering-shell`, `unproven.scoped-admin-change`, `unproven.history-bound-on-mainnet`, `unproven.budget-shell`, and 19 assumed ones, `rule.cond-room`, `rule.functions-room`, `rule.lower-room`, `rule.empty-batch`, `rule.no-creation-vocabulary`, `rule.ledgers-start-at-1`, `trusted.rustc`, `trusted.charon`, `trusted.aeneas`, `trusted.aeneas-result-loop`, `trusted.aeneas-library-items`, `trusted.lean-kernel`, `trusted.denotations`, `trusted.document-meaning`, `trusted.soroban-models`, `trusted.soroban-host`, `trusted.sdk-items`, `trusted.facade-conversion`, `trusted.host-harness`. The assumed claims are hypotheses of the model (bounded room, the empty batch, no creation vocabulary, ledgers from 1) and the trusted items of the trust base in §9.

Table 9: The ten attenuation steps (`crates/rule-eval/src/attenuate.rs` in rule-core). Nothing else is an attenuation step. Document conditions lower to these rule-core conditions: `is-self`, `address-eq` and `u32-eq` to `ArgEq`, `string-in` to `ArgIn`, `string-prefix` to `ArgStrPrefix`. `ArgI128AtMost` appears in a lowered document only as the type check on a capped transfer's amount, with the largest `i128` as its bound, so a document cannot bound one call's amount; only an attenuation step can set a smaller maximum (rule-core `lowering.rs`).

Rule part	Steps that may narrow it	Refused
Rule set	`drop_rule` (later rule indices shift down)	none
Functions	`remove_functions`, `restrict_functions`	none
Arguments	`add_cond` (`ArgEq`, `ArgIn`, `ArgStrPrefix`, `ArgI128AtMost`)	none
Signers	`remove_signers` (threshold kept), `raise_threshold`	a lower threshold
Cap	`add_cap` to an uncapped rule, `lower_cap_limit`, `lengthen_cap_period`	a second cap, a non-positive limit, a zero period, a higher limit, a shorter period
Expiry	`lower_valid_until` (add an expiry or move it earlier)	a later expiry
Target, generation	no step	none

Table 10: Four counters, each with its own trigger; Figure 3 shows the first two in the reference run.

Counter	Lives in	Advances on	Used for
Grant generation	Account	Each completed recovery, nothing else	Stamping grants; refusing stale ones
Recovery generation	Account	Every recovery-configuration change (enrollment, reconfiguration, removal, each side of a controller switch), each completed recovery, each executed upgrade	Making scheduled upgrades stale
Revision	Account	Every `apply_account_doc` and executed upgrade	The optional expected-revision check on account-document writes
Epoch	Recovery controller	Every configuration change, completed recovery and executed account upgrade	Invalidating older evidence and attempts

C Comparison Sources and Open Items

Figure 7 replays the walkthrough's recorded calls; one of its panels is the per-call run of §8. Figure 8 follows one guarded call through both accounts, phase by phase, as read from source. Figure 9 spells, in both systems, the game grant that the testnet preview of Nido installed, not grant 3; its marks are: ① both select one rule per context, so a transfer to the game is allowed without a `guess`; ② Perch keeps the window in the rule, while OZ-PB turns the Intent's `cap` into a per-call

Table 11: Shell obligations that carry the theorems to the account. Each obligation is what the account shell must do for a statement about the evaluator to hold of the deployed account.

	Obligation	Status	Source
O1	The shell authenticates every claimed signer	tested	`gate.rs:106-113`
O2	Selection and renumbering carry the evaluator's theorems to the account	argued and tested	`gate.rs:14-16`
O3	The generation is passed in, advances only on completion, and refuses on overflow	tested; rule-core lists it as unproven against Perch `dff2a73`	rule-core `claims.yaml:2901`
O4	The excluded set matches the theorem's X, and the recheck runs	tested	`lib.rs:1225-1234`
O5	Compiler outputs satisfy `Doc.Valid`	unproven	rule-core `claims.yaml:2928`
O6	Spending records persist correctly	tested	`gate.rs:114-121`
O7	Context conversion is correct	tested	`crates/perch-rule-eval/src/convert.rs`
O8	The shell installs exactly the lowered rules, and ordinary authorization cannot select the recovery rule	tested; rule-core lists it as unproven	rule-core `claims.yaml:2940`

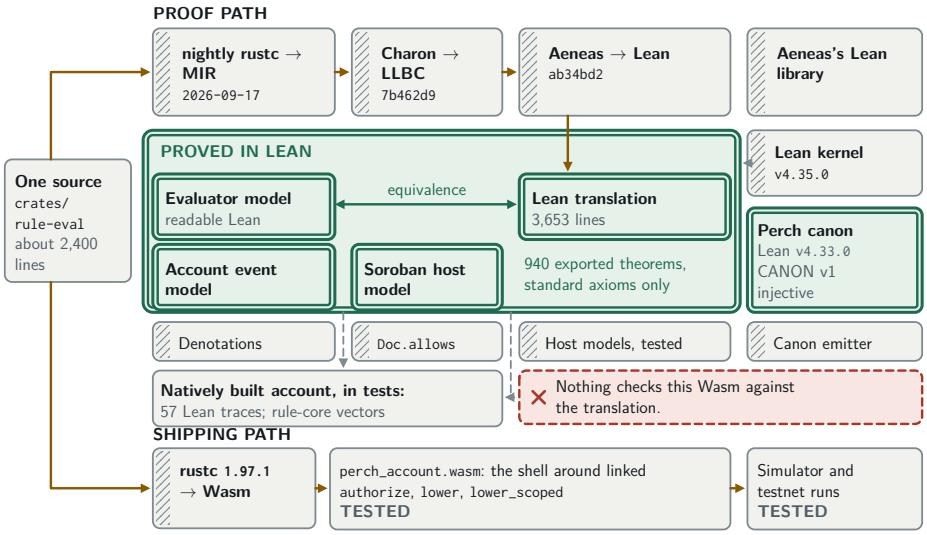


Fig. 5: One source, two paths. Lean's kernel checks theorems about the Aeneas translation of the MIR that nightly rustc produces. The account ships WebAssembly from rustc 1.97.1, and nothing checks one against the other. Double border: proved. Single border: tested. Hatched: trusted. Inside the double border are the objects the theorems are about; that each model matches its subject is trusted (hatched row). Dashed connectors are the tests that bridge the two paths.

`lte` and attaches `spending_limit`; ③ Perch installs both rules in one transaction, OZ-PB one rule per `install_policy` call; ④ Perch's rule holds a passkey as an external signer, while OZ-PB writes every rule signer as delegated, and the interpreter refuses external signers (error 212); ⑤ in Perch, `not-after-ledger` N

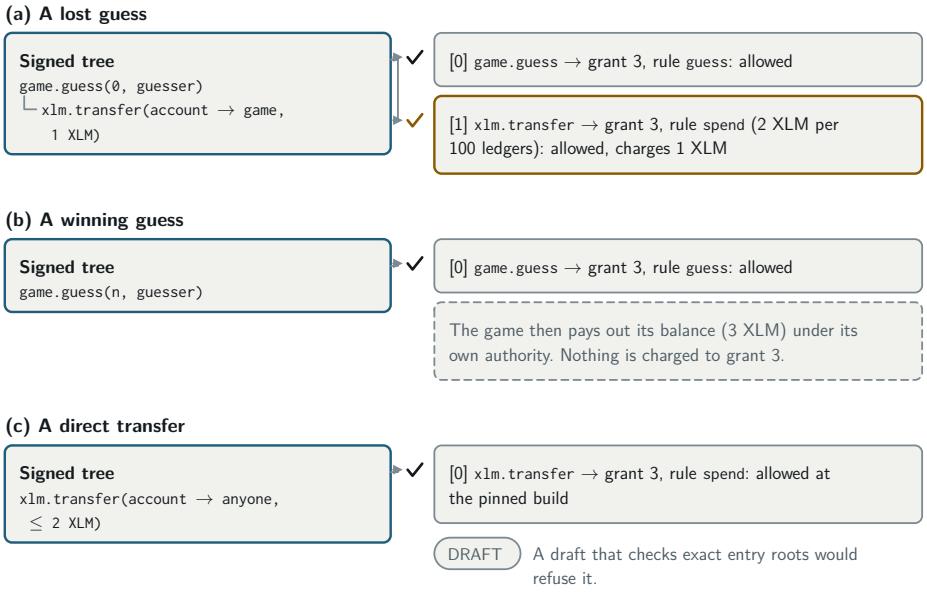


Fig. 6: One authorization, several contexts, using grant 3. The authorization payload names one installed rule per context. (a) A lost guess: the game's `guess` and the transfer it makes from the account; the capped rule charges the transfer. (b) A win authorizes only the root, and the game pays from its own balance. (c) A direct transfer within the cap, with no game call, is also allowed, because each context is checked alone. Draft changes after the pinned build (Perch #69 and rule-core #17) would let a rule require its entry's root (§4).

authorizes through N − 1, while OpenZeppelin's `valid_until` authorizes through itself. Terms differ between the systems: *scope* is a rule's target or an installed grant in Perch and a context rule's kind in OpenZeppelin; Perch has no *policy* contracts, and its earlier build's *interpreter* is a Perch 0.5.1 policy contract; *execute* is Perch's owner call-through and OZ-PB's adapter entry point; *recovery* replaces credentials in Perch and moves custody funds in OZ-PB's Prime.

Table 13 gives all twenty dimensions. Each cell cites its source at the pins: Perch f0275f4 unless marked, rule-core cbabe59, oz-policy-builder 43b13ed (`43b13ed1d9c6662afd74f446006e69bfcf21259b`, 2026-10-30), and OpenZeppelin `stellar-contracts` v0.8.0 (ddc327e) and v0.7.2 (a9c4216). The mark records how a cell was checked: V, checked in source at the pin by the authors; R, cited from source by an internal reader and not re-read by the authors; U, not verified.

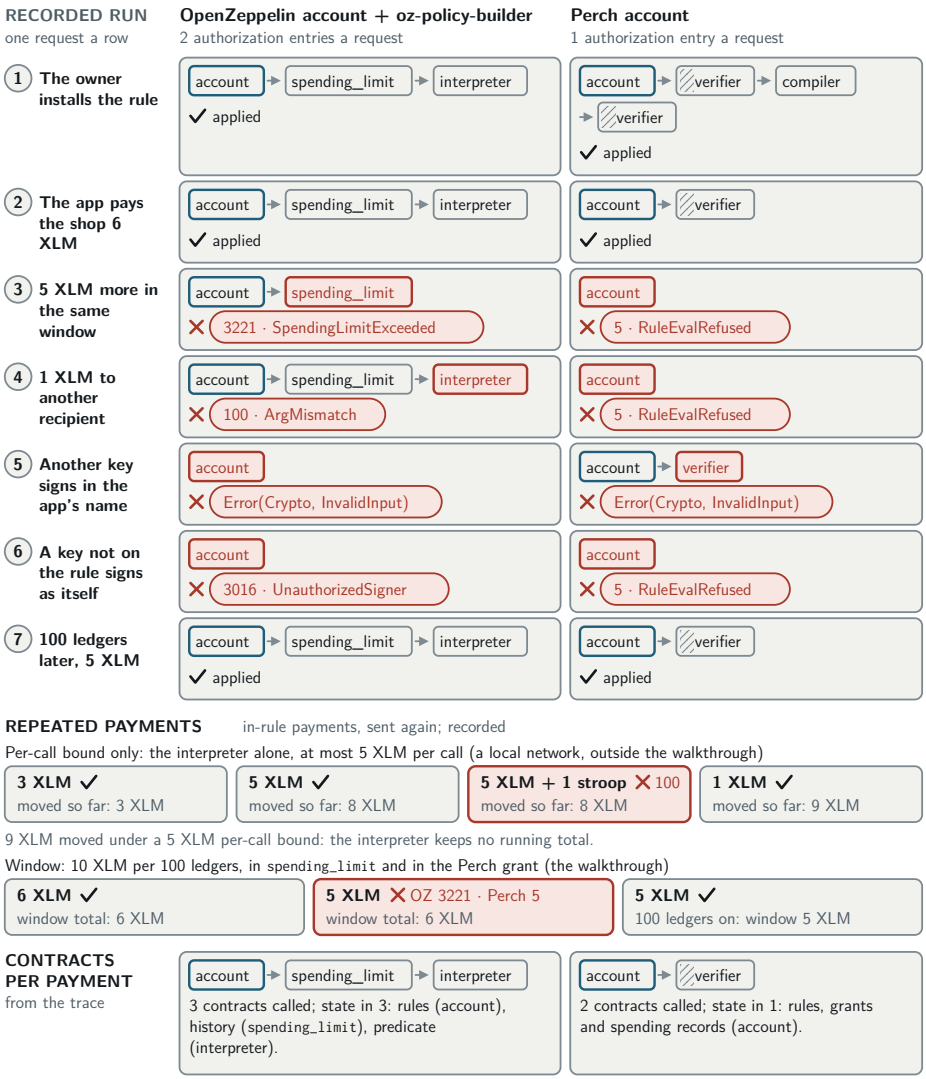


Fig. 7: One task in both designs, replayed from the recorded run. Each row is one request of the walkthrough (Table 1); for each design it lists, in order, the contracts called while the account authorized the request, and where the request was refused. Below the rows, one panel repeats in-rule payments: under a per-call bound alone, an OpenZeppelin account whose rule carries only the interpreter, run on a local network outside the walkthrough, where three sends moved 9 XLM under a 5-XLM bound; and under a window, OpenZeppelin's `spending_limit` or a Perch grant, from the walkthrough. The last panel lists the contracts each design called for one payment and where its decision state lives. The OpenZeppelin side's second authorization entry is its delegated signer's, because oz-policy-builder takes only delegated signers. Calls and codes are recorded, not drawn from source.

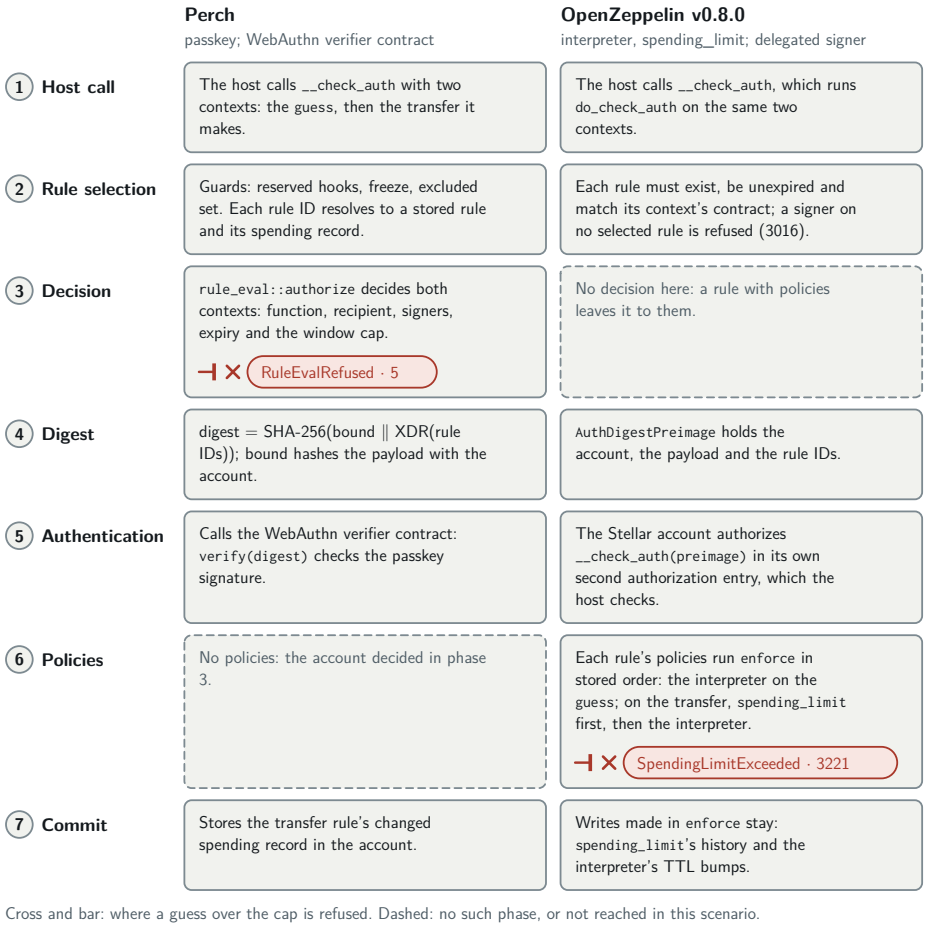


Fig. 8: One guarded call in two accounts. One guarded call, phase by phase, through `__check_auth`, in three scenarios (a lost guess, a guess over the cap and a direct transfer): under Perch, and under an OpenZeppelin account whose rules carry oz-policy-builder's `policy-interpreter` and OpenZeppelin's `spending_limit`. The Perch signer is a passkey checked by a verifier contract. oz-policy-builder's rules accept only delegated signers; here the signer is a Stellar account whose own authorization entry the host checks. The OpenZeppelin account runs a rule's policies in the rule's stored order, which for policies given at creation is their address order. A cross marks where each design refuses a guess over the cap: Perch before any signer is authenticated, the OpenZeppelin account in `spending_limit` after authentication. Dashed: no such phase in that design, or not reached in the scenario. Read from source at Perch f0275f4, OpenZeppelin `stellar-contracts` v0.8.0 and oz-policy-builder 43b13ed; Table 1 gives the measured run of a related task.

(a) Perch grant document

```
{ "version": 1,
  "network": "Test SDF Network ; September 2015",
  "signers": [
    { "id": "app", "verifier": "C...WEBAUTHN", ④
      "key": "...65-byte passkey key, hex..." } ],
  "rules": [                                    ③
    { "name": "guess",
      "scope": { "type": "contract",
                 "address": "C...GAME" },
      "principals": { "type": "all",
                      "signers": ["app"] },
      "functions": ["guess"],
      "not-after-ledger": N },                  ⑤
    { "name": "guess-xlm",                      ①
      "scope": { "type": "contract",
                 "address": "C...XLM" },
      "principals": { "type": "all",
                      "signers": ["app"] },
      "functions": ["transfer"],
      "args": [ { "index": 1,
                  "pred": { "type": "address-eq",
                            "address": "C...GAME" } } ],
      "cap": { "limit": "20000000",             ②
               "period-ledgers": 17280 },
      "not-after-ledger": N } ] }               ⑤
```

(b) oz-policy-builder Intent

```
{ "calls": [
  { "contract": "C...GAME", "fn": "guess" },
  { "contract": "C...XLM", "fn": "transfer", ①
    "where": [
      { "constraint": "pin", "at": [1],
        "to": "C...GAME" },
      { "constraint": "cap", "at": [2],       ②
        "max": "20000000" } ] } ] }
```

(c) XLM rule: wire predicate and install arguments

```
["and", [["eq", ["call_contract"], "C...XLM"],
         ["eq", ["call_fn"], "transfer"],
         ["eq", ["call_arg", 1], "C...GAME"],
         ["lte", ["call_arg", 2],           ②
                 {"i128": "20000000"}]]]

{ "network": "testnet",
  "smartAccount": "C...ACCOUNT",
  "sourceAccount": "G...OWNER",
  "intent": { "...": "(b)" },
  "contract": "C...XLM",                     ③
  "signers": ["G...APP"],                    ④
  "validUntilLedger": N - 1,                 ⑤
  "spendingLimit": { "amount": "20000000", ②
                     "periodLedgers": 17280 } }
```

Fig. 9: One intent, two spellings. The intent: the app key may call `guess` on the game, and may send XLM only to the game, at most 2 XLM per 17,280 ledgers, through ledger $N - 1$. (a) Perch: one grant document, installed by one `apply_grant` call. (b) oz-policy-builder: an Intent, lowered to one predicate per contract. (c) The XLM rule's predicate in wire form, and the `install_policy` arguments that attach OpenZeppelin's `spending_limit`; a second call installs the game rule. `C...`, `G...` and N are placeholders; `{ "...": "(b)" }` stands for the Intent in (b), passed as `intent`. Read from source; these two oz-policy-builder installs were not run.

Table 12: Perch and oz-policy-builder, side by side. Twelve of the twenty dimensions; every cell is read from source at Perch f0275f4, oz-policy-builder 43b13ed and OpenZeppelin `stellar-contracts` v0.8.0. The full comparison table gives a source for each cell.

Dimension	Perch f0275f4	oz-policy-builder 43b13ed on an OpenZeppelin account
Where the decision runs	Inside the account: the linked evaluator decides	In attached policy contracts, called by the account after authentication
Order of checks	Decide (caps included), then authenticate	Select rules (existence, scope, expiry) and refuse unlisted signers, authenticate, then run each policy in the rule's stored order
Where decision state lives	The account only	The account, the interpreter and `spending_limit`, each keyed by (account, rule)
Conditions on one call	Target, functions, five conjunctive argument conditions, threshold, expiry	A predicate language with and/or, comparisons, quantifiers, arithmetic, time and reads
Relating calls in one transaction	Each context alone	One predicate can relate the calls of an adapter batch
External state in the decision	None	Up to 8 cross-contract reads per `enforce`
Rolling spending cap	In the evaluator, window $(L - P, L]$; proved for the evaluator from empty records	Not in the interpreter; OpenZeppelin's optional `spending_limit` beside it, on `transfer` only, same window boundary
Threshold and signer kinds	Threshold in the rule; external (passkey, Ed25519) and delegated signers	Threshold through an attached policy; delegated signers only on policed rules
What a signature binds	External: account and rule IDs; delegated: account only	v0.8.0: account and rule IDs for both kinds
Ending authority in bulk	One counter makes every earlier grant refuse at recovery	Rules removed one at a time; a policed rule is pinned to its exact signer set
Assurance	Kernel-checked proofs about the evaluator's Lean translation, not its Wasm; shell tested, partly on Wasm; no audit completed	Tests, partly on the shipped Wasm; STRIDE register; external audit in progress
Deployment	Testnet preview (earlier design on mainnet)	Interpreter on testnet; no mainnet pin

Table 13: Perch and oz-policy-builder, all twenty dimensions with sources. Perch paths are at f0275f4, oz-policy-builder paths at 43b13ed, OpenZeppelin (OZ) paths at `stellar-contracts` v0.8.0 unless marked. Mark: V checked in source at the pin by the authors; R cited from source by an internal reader and not re-read by the authors; U not verified. Nothing here was measured; the policy order was also observed in the walkthrough.

Dimension	Perch	oz-policy-builder on an OpenZeppelin account	Mark
Where the decision runs	Linked `rule_eval::authorize` inside `__check_auth` (`gate.rs:93-105`)	The account selects the rule and authenticates, then calls each attached policy's `enforce`; the shared interpreter evaluates the stored predicate (OZ `storage.rs:629-690`; `policy-interpreter/src/lib.rs:192-277`)	V

Dimension	Perch	oz-policy-builder on an OpenZeppelin account	Mark
Order of checks	Whole rule decision, caps included, before authentication; a refused batch calls no verifier (gate.rs:93-113)	Rule existence, scope and expiry, then a refusal of any signer on no selected rule, before authentication; then each attached policy's enforce (predicate, spending total, threshold) in the rule's stored order: address order for the policies it was created with, then any added later (OZ storage.rs:629-689, 830, 1304, 1588-1596; observed in the walkthrough)	V
Where decision state lives	Account storage only: rules, grants, spending records, generation (lib.rs:597-636)	Account (rules, signers); interpreter, one entry per (account, rule) (policy-interpreter/src/storage.rs:167-188); spending_limit, history per (account, rule) (OZ policies/spending_limit.rs)	V/R
Conditions on one call	Target, function list, five conjunctive argument conditions, threshold, expiry; no per-call amount bound (perch-ir/src/doc.rs:281-304; rule-core lowering.rs:17-20)	And/or, six comparisons, in/nin, all/any/sum over vectors, scaled arithmetic, ledger time now, nested call_path, read (policy-interpreter/src/dsl.rs:84-158); per-call lte on amounts	V
Relating calls in one transaction	None: each context is checked alone against its rule (Figure 6(c)); draft changes after the pinned build (Perch #69 and rule-core #17) add entry roots	One predicate over the execution adapter's batch argument can relate its calls (docs/stellar.md:350-352; policy-interpreter/README.md:390-397)	V
External state in the decision	None; reads account storage only	read calls other contracts during enforce, up to the instance's max_reads (8 on the testnet pin) (dsl.rs:409-423; config.ts:66-73); same-transaction manipulation is open at High (T-05, stride-threat-model.md:397)	V
Rolling spending cap	In the evaluator; window (L − P, L]; proved from empty records (T6); at most 907 spends per window on chain (docs/recovery/budgets.md:604-611)	Not in the interpreter (dsl.rs:65-72). OpenZeppelin spending_limit beside it: same window boundary, at most 815 entries, transfer only (OZ spending_limit.rs:9-12, 169; install.ts:957-967); the oz-policy-builder authors did not audit it (evidence/README.md:157)	V
Threshold and signer kinds	Threshold inside the rule, counting distinct identities (T3); External (Ed25519, WebAuthn) and Delegated (CAP-0071) signers	A rule with any policy accepts one signer unless OpenZeppelin's threshold policy is attached; the builder refuses that shape unless anyOne (tools.ts:590). Policed rules accept Delegated signers only (lib.rs:455-462, error 212)	V
What a signature binds	External: account and rule IDs. Delegated: account, not rule IDs (gate.rs:106-113; auth.rs:47-61)	v0.8.0: account and rule IDs for both kinds; an external signer signs the hash of AuthDigestPreimage, a delegated signer authorizes it through require_auth_for_args (OZ storage.rs:150-175, 396-412). 0.7.2-line: rule IDs, not the account, for External with a legacy Address credential (OZ v0.7.2 storage.rs:492-495)	V
Ending authority in bulk	A completed recovery advances the grant generation, and every earlier grant refuses in that transaction (T10); otherwise one grant at a time	Rules are removed one at a time. A policed rule is pinned to its exact signer set (error 204 after any change). Removing a rule with only rule 0's signatures leaves the interpreter's entry for it until the entry archives (docs/stellar.md:548-562); a rule that follows it keeps enforcing it (policy-synth README, revoke_policy)	V

Dimension	Perch	oz-policy-builder on an OpenZeppelin account	Mark
Assurance	Lean proofs of the evaluator over its translation (§9); shell tested; no external audit completed	440 interpreter tests (26 against OpenZeppelin's 0.8.0 Wasm), parity corpus, worst-case suite on Wasm, STRIDE model, mutation testing of one path; no formal model found in the tree (a search, not a proof of absence); pull request #78 hardened the off-chain packages after an external penetration test; external audit of v5.2.0 in progress since 26 October 2026, with the OpenZeppelin account and policies out of scope (README.md:57-59, 166-171; audit-overview.md:15-20, 76-85; evidence/README.md:23)	V
Deployment	This design: testnet preview. Earlier design c135aca: testnet and mainnet	Interpreter pinned on testnet only; no mainnet pin (config.ts:57-80). OpenZeppelin v0.7.2 policies on mainnet (evidence/README.md:154-157)	V
Rules with no signer	Refused, UnguardedRule (rule_set.rs:53-55)	Allowed when bound to an execution adapter; anyone may then run what the predicate permits (lib.rs:210-223; architecture.md:149)	V/R
Rules shared across accounts	None; grant rules are copied into the account at install	A rule may follow another account's rule, whose admins can change the predicate (E-38, Medium, accepted) (lib.rs:76-84; audit-overview.md:243-278)	V
Account-key recovery	Guardian or ZK recovery replaces credentials (§7)	None; Prime's "recovery" moves custody funds to a fixed address (architecture.md:81-94)	V
Install ordering	apply_doc takes an optional expected revision; apply_scope takes none (lib.rs:1000-1006; docs/account-interface.md:96-135)	install_nonce must be the stored nonce plus one (lib.rs:144-145)	V
Authoring tools	perch-js builder (packages/perch-js/README.md:31-44); on-chain compiler; offline z3 checks in perch-analyze (crates/perch-analyze/src/lib.rs:1-27)	Recorder, synthesiser, verifier with minimality, review renderer, unsigned install builder, overlap scan; nine tools over a CLI and an MCP server (README.md:13-24)	V/R
Who can change deployed code	Account: the owner, after 120,960 ledgers (perch-recovery-interface/src/account.rs:15). Compiler, verifiers, controller: no admin (docs/recovery/spec.md:100). Registry: admin-upgradeable (perch-constructorless-registry/src/lib.rs:105-108)	Interpreter, adapter, gate and seat: no admin and no upgrade path (architecture.md:153). Oracles on the safe list are admin-upgradeable (config.ts:215). OpenZeppelin account and policies: not examined	V/R/U
Keeping state alive	Rule set extended on every authorization; grants on read; spending records to the maximum on write; renew omits grants (gate.rs:64, 114-119; lib.rs:936-959)	Interpreter entry bumped on a permit when fewer than 100 ledgers remain; instance and read targets need a keep-alive job (lib.rs:256-265; docs/stellar.md:304-308)	V/R
What a refusal reports	StaleScope (3) or RuleEvalRefused (5); a wallet re-runs the evaluator off chain for the reason	A specific deny code per reason (storage.rs:27-86)	V

Comparator artifacts. oz-policy-builder pins one interpreter, on testnet only (mainnet: none pinned), at CDER6P73JWMIVD3X6TGGOQIDAH3DLU7OKRTLT4ULPTZ3QJJ3QUGTK7CM, running Wasm 6704996f626904a4e33b95bda17b73963fc9cf502ffdfdbe9d85b4b10288e0d4 with deploy-time limits reads 8, leaves 400, bytes 24,576, elems 32, in 32, depth 5; its execution adapter runs bf85432c.... It signs for two OpenZeppelin account builds: the v0.8.0 build 8286422f... and the v0.7.2-line

Table 14: Disclosed open items in both systems. Items each project documents about itself. oz-policy-builder rates its items in a STRIDE register; Perch keeps none, so its rows give the status its documents state. H-1/E-36 and E-38 arise from features Perch lacks; their absence in Perch is not a security property of Perch.

System	Item	Status	Source
Perch	The completion marker is consumed by the target's canonical identity, not its exact bytes	open	issue #15
Perch	Delegated signers do not bind rule IDs	by design (CAP-0071)	`docs/recovery/cap-0071.md:28`
Perch	Replacing a capped rule, or changing its cap, starts an empty window	documented	`docs/DEFERRED.md:298-324`
Perch	A rule edited in place between signing and inclusion decides under its new contents	documented	`docs/DEFERRED.md:227-262`
Perch	Grant installs take no expected version	stated	`lib.rs:1000-1006`
Perch	One key is admin and manager of `perch/verifiers` on testnet and mainnet; the admin can replace the registry's code at once	deployment fact	`deployments/*.json:22-23`
oz-policy-builder	H-1/E-36: a keyed door on a no-gate adapter without the grants pin	High, residual Medium; tooling-only guard	`audit-overview.md:165-177`
oz-policy-builder	E-38: a keyless follower's fence can be loosened by the followed rule's admins	Medium, accepted	`stride-threat-model.md:400`
oz-policy-builder	T-05: same-transaction read manipulation	High where a rule uses `read`	`stride-threat-model.md:397`
oz-policy-builder	T-19: the review reaches the owner through the agent	High until the owner side runs it	`stride-threat-model.md:398`

build 91a2cd56…, which has no recorded source. OpenZeppelin's policies are deployed at v0.8.0, deployed 26 October 2026 on testnet and v0.7.2, deployed 9 June 2026 on mainnet; `spending_limit` keeps at most 815 history entries at v0.8.0 and 1,000 at v0.7.2, and refuses with 3221. The interpreter's refusal codes include 100 (an argument mismatch), 202 (an install nonce that is not the next), 204 (a changed signer set), 208 (a predicate-hash mismatch), 210 (no authenticated signer), 212 (an external signer) and 218 (a rule that reaches the account's administration). An install's authorization window is 300 ledgers by default, at most 17,280 ledgers. Its mutation testing killed 31 of 32 killed mutants on one path.

Comparative method. Table 1 runs one task in both designs in one soroban-sim harness, without exact metering, with Ed25519 keys (a delegated Stellar account on the OpenZeppelin side); it is not workload W1. We propose, and have not run, five workloads under one harness with exact metering and one host version, reporting CPU instructions, memory, ledger I/O entries and bytes, fees, and cross-contract calls counted from diagnostic events: W1, one capped transfer; W2, a losing guess, a root and its child; W3, one grant install; W4, ending authority over k grants, one recovery completion against k calls to `remove_`

`context_rule`; and W5, storage after n transfers. Beyond Table 1's results, we report no comparative cost.

References

1. Birgisson, A., Politz, J.G., Erlingsson, Ú., Taly, A., Vrable, M., Lentczner, M.: Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In: Network and Distributed System Security Symposium (NDSS) (2014), https://research.google/pubs/macaroons-cookies-with-contextual-caveats-for-decentralized-authorization-in-the-cloud/
2. Biscuit authorization token specification. biscuitsec.org, https://www.biscuitsec.org/
3. Buterin, V., Weiss, Y., Tirosh, D., Nacson, S., Forshtat, A., Gazso, K., Hess, T.: Erc-4337: Account abstraction using alt mempool. Ethereum Improvement Proposals, https://eips.ethereum.org/EIPS/eip-4337
4. cedar-spec, revision a4c5a89cd15637afc1dab39ef0c89baf31c7ebdf. GitHub, cedar-policy/cedar-spec (2026), https://github.com/cedar-policy/cedar-spec/tree/a4c5a89cd15637afc1dab39ef0c89baf31c7ebdf, lean formalization of Cedar and differential testing against its Rust implementation; Lean 4.35.0
5. Certora: User guide for sunbeam. Certora documentation, https://docs.certora.com/en/latest/docs/sunbeam/usage.html, contract specification and Wasm verification workflow
6. Cutler, J.W., Disselkoen, C., Eline, A., He, S., Headley, K., Hicks, M., Hietala, K., Ioannidis, E., Kastner, J., Mamat, A., McAdams, D., McCutchen, M., Rungta, N., Torlak, E., Wells, A.M.: Cedar: A new language for expressive, fast, safe, and analyzable authorization. Proceedings of the ACM on Programming Languages (OOPSLA) (2024), https://www.amazon.science/publications/cedar-a-new-language-for-expressive-fast-safe-and-analyzable-authorization
7. Disselkoen, C., Eline, A., He, S., Headley, K., Hicks, M., Hietala, K., Kastner, J., Mamat, A., McCutchen, M., Rungta, N., Shah, B., Torlak, E., Wells, A.: How we built cedar: A verification-guided approach. In: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion) (2024), https://arxiv.org/abs/2407.01688
8. Ellison, C., Frantz, B., Lampson, B., Rivest, R., Thomas, B., Ylönen, T.: Spki certificate theory. RFC 2693 (1999), https://www.rfc-editor.org/rfc/rfc2693
9. Gnosis Guild: Zodiac roles modifier: conditions. Zodiac documentation, https://docs.zodiac.eco/developers/roles/conditions
10. Guess the number fixture, revision df80831ac6bc2a2110cab15654726456b53cc8f3. GitHub, nidohq/guess-the-number (2026), https://github.com/nidohq/guess-the-number/tree/df80831ac6bc2a2110cab15654726456b53cc8f3, tested source commit, published in draft pull request nidohq/guess-the-number#7 and unmerged at artifact intake; its exact build-input archive is in the paper's source download
11. Hardy, N.: The confused deputy (or why capabilities might have been invented). ACM SIGOPS Operating Systems Review **22**(4) (1988), https://doi.org/10.1145/54289.871709
12. Ho, S., Protzenko, J.: Aeneas: Rust verification by functional translation. In: Proceedings of the ACM on Programming Languages (ICFP) (2022), https://arxiv.org/abs/2206.07185

13. Isailovic, L., Rein, D., Finlay, D., Chiang, D., Makarov, F., Gomes, P., Swenberg, C., Rosario, L., Bowman, I., Smale, J.: Erc-7715: Request permissions from wallets. Ethereum Improvement Proposals (2024), https://eips.ethereum.org/EIPS/eip-7715, draft
14. Josefsson, S., Liusvaara, I.: Edwards-curve digital signature algorithm (eddsa). RFC 8032 (2017), https://www.rfc-editor.org/rfc/rfc8032
15. Kozhevin, D.: Cap-0046-11: Soroban authorization framework. Stellar Core Advancement Proposals, https://github.com/stellar/stellar-protocol/blob/master/core/cap-0046-11.md, final
16. Kozhevin, D.: Cap-0071: Authentication delegation and address-bound soroban credentials. Stellar Core Advancement Proposals (2025), https://github.com/stellar/stellar-protocol/blob/master/core/cap-0071.md, final; protocol 27
17. McPeck, R., Finlay, D., Dawson, R., Chiang, D.: Erc-7710: Smart contract delegation. Ethereum Improvement Proposals, https://eips.ethereum.org/EIPS/eip-7710, interface proposal (Draft)
18. National Institute of Standards and Technology: Module-lattice-based digital signature standard. FIPS 204 (2024), https://csrc.nist.gov/pubs/fips/204/final
19. OpenZeppelin: Stellar smart account contracts, v0.8.0 (ddc327eb92d63709436da3a8455f641c2d5476c5). GitHub, OpenZeppelin/stellar-contracts, packages/accounts (2026), https://github.com/OpenZeppelin/stellar-contracts/tree/ddc327eb92d63709436da3a8455f641c2d5476c5/packages/accounts/src, context rules, signers and policies
20. Perch deployment manifests, deployments/testnet.json and deployments/mainnet.json at f0275f4. GitHub, bitswell-hq/stellar-registry___perch (2026), https://github.com/bitswell-hq/stellar-registry__perch/tree/f0275f41c5ab47c903bbfe9986b4bf3239c01b11/deployments
21. Rundgren, A., Jordan, B., Erdtman, S.: Json canonicalization scheme (jcs). RFC 8785 (2020), https://www.rfc-editor.org/rfc/rfc8785
22. Runtime Verification: Komet, revision a078e296621a4f52fd1497fed64435e9948cc830. GitHub, runtimeverification/komet, https://github.com/runtimeverification/komet/tree/a078e296621a4f52fd1497fed64435e9948cc830, pinned contract-semantics source
23. Saltzer, J.H., Schroeder, M.D.: The protection of information in computer systems. Proceedings of the IEEE **63**(9) (1975), https://doi.org/10.1109/PROC.1975.9939
24. Schneider, F.B.: Enforceable security policies. ACM Transactions on Information and System Security **3**(1) (2000), https://doi.org/10.1145/353323.353382
25. soroban-sim, revision d02938430f0ac41f34e5e3e4c8c48d85b871bd68. GitHub, theahaco/soroban-sim (2026), https://github.com/theahaco/soroban-sim/tree/d02938430f0ac41f34e5e3e4c8c48d85b871bd68, portable host and session driver; host 29.0.2
26. Stellar Development Foundation: Custom account example. Stellar developer documentation, https://developers.stellar.org/docs/build/smart-contracts/example-contracts/complex-account, soroban authorization payloads and contexts
27. Ucan specification. GitHub, ucan-wg/spec, https://github.com/ucan-wg/spec
28. Untangled Finance: Oz policy builder, revision 43b13ed1d9c6662afd74f446006e69bfcf21259b (v5.2.0 line). GitHub, untangledfinance/oz-policy-builder (2026), https://github.com/untangledfinance/oz-policy-builder/tree/43b13ed1d9c6662afd74f446006e69bfcf21259b, policy synthesiser, policy-interpreter contract, execution adapter, custody gate and Prime seat
29. W3C: Web authentication: An api for accessing public key credentials, level 2. W3C Recommendation (2021), https://www.w3.org/TR/webauthn-2/

30. Wyndham, W.: soroban-rule-core, revision cbabe594ae2df408991e825e56ddd618ea29541b. GitHub, bitswell/soroban-rule-core (2026), https://github.com/bitswell/soroban-rule-core/tree/cbabe594ae2df408991e825e56ddd618ea29541b, rust evaluator, model, translation and proof trust inventory
31. Wyndham, W., Engelman, E.: Perch source, revision f0275f41c5ab47c903bbfe9986b4bf3239c01b11. GitHub, bitswell-hq/stellar-registry___perch (2026), https://github.com/bitswell-hq/stellar-registry___perch/tree/f0275f41c5ab47c903bbfe9986b4bf3239c01b11, account, compiler, recovery, verifiers and canonical-document proof
32. Xu, Z., Stoller, S.D.: Mining attribute-based access control policies. IEEE Transactions on Dependable and Secure Computing **12**(5) (2015), https://doi.org/10.1109/TDSC.2014.2369048
33. zeroknots, Kopp, K., Lee, T., Veiga, F., Rhinestone, Biconomy: Erc-7579: Minimal modular smart accounts. Ethereum Improvement Proposals, https://eips.ethereum.org/EIPS/eip-7579, draft